计算机操作系统

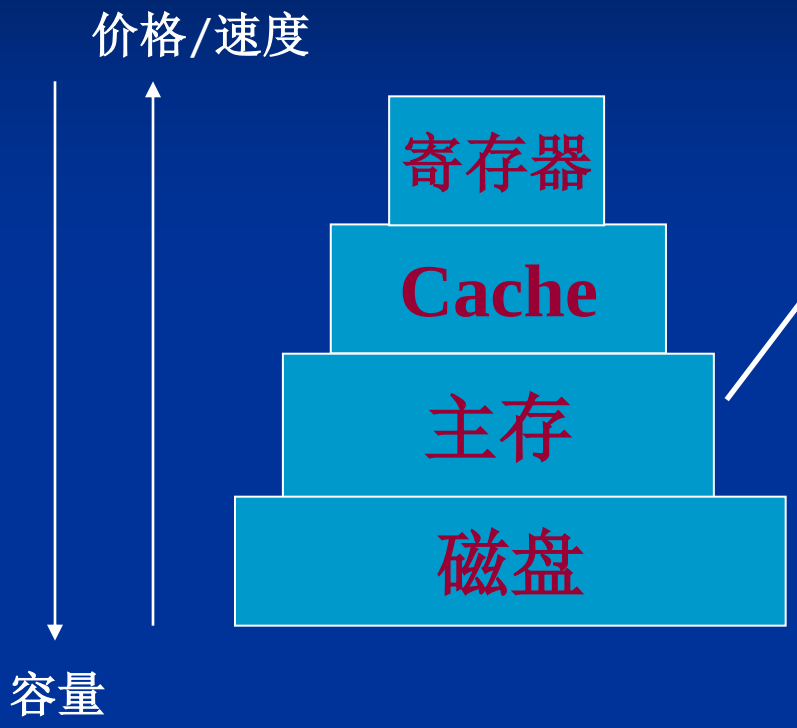
Operating Systems

田卫东

March, 2014

第4章 存储管理

存储体系概述



- 由存储单元（字节或字）组成的一维连续的地址空间，简称内存空间。
- 用来存放当前正在运行程序的代码及数据，是程序中指令本身地址所指的、亦即程序计数器所指的存储器
- 性能“瓶颈”：关键、紧张
- 帕金森定律：

  内存多大，程序多长

4.1 程序的装入和链接

4.1.1 程序的装入

- 绝对装入方式

  编程时，直接确定内存位置；

  特点：

  不灵活，不支持多道程序设计技术；

4.1 程序的装入和链接

4.1.1 程序的装入

- 可重定位装入方式

虚拟技术：编程时使用逻辑地址，运行时使用物理地址；

装入时，进行地址重定位；

特点：

灵活，支持多道程序设计技术；

- 逻辑地址（相对地址，虚地址）

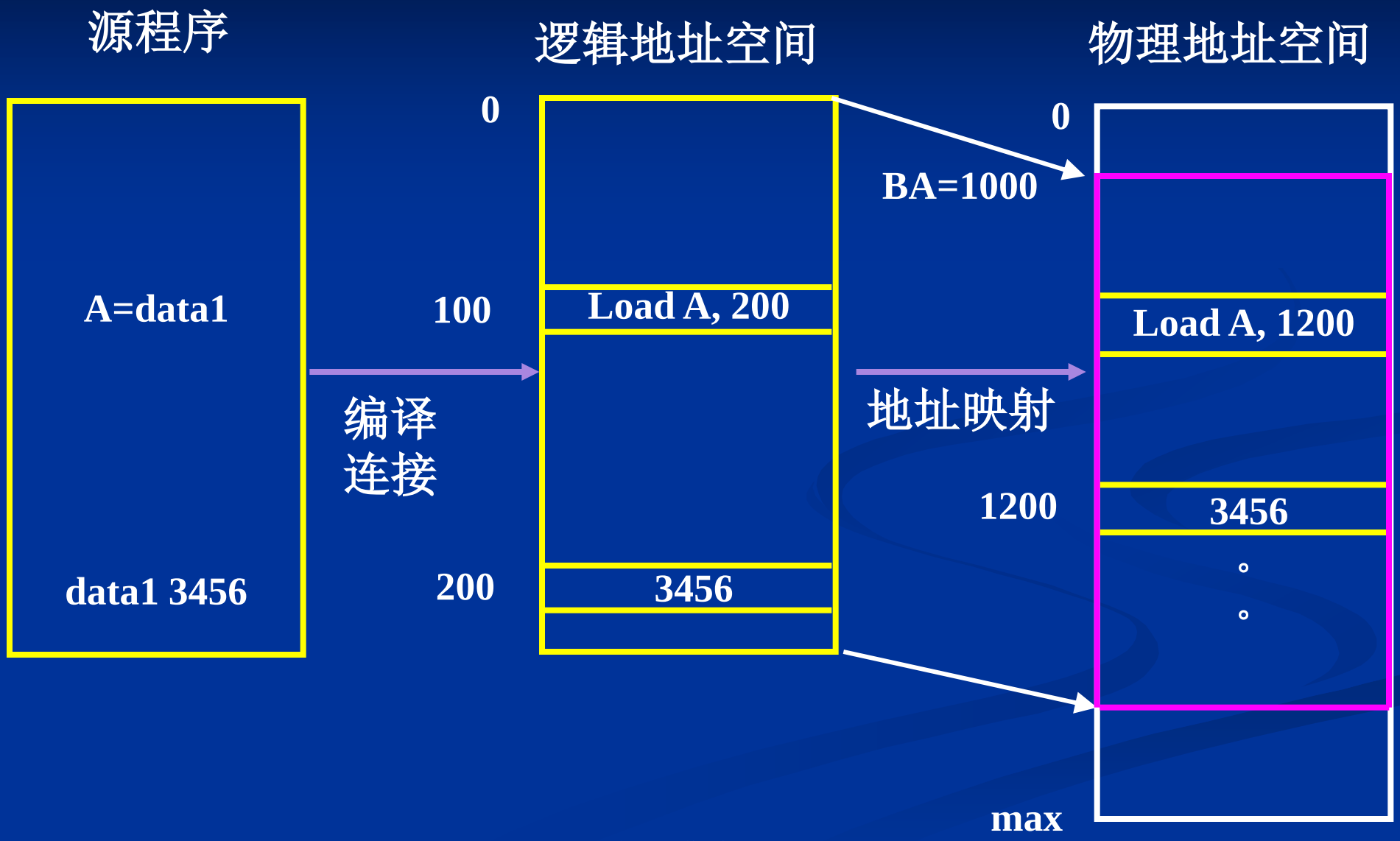
  用户程序经过汇编或编译后形成目标代码，目标代码朱红所有地址采用相对地址形式:

  - 首地址为0;
  - 其余指令中地址以相对于首地址的字节偏移为地址;

  不能用逻辑地址在内存中读取信息。

- 物理地址（绝对地址，实地址）

  内存中存储单元的地址，可直接寻址。


源程序
逻辑地址空间
物理地址空间
A=data1
data1 3456
0
100
200
Load A, 200
3456
编译
连接
BA=1000
0
Load A, 1200
1200
3456
地址映射
max

地址重定位（地址映射）

为了保证CPU执行指令时可正确访问存储单元，需将用户程序中的逻辑地址转换为运行时由机器直接寻址的物理地址，这一过程称为地址映射。

逻辑地址（相对地址，虚地址）



物理地址（绝对地址，实地址）

- 静态地址重定位

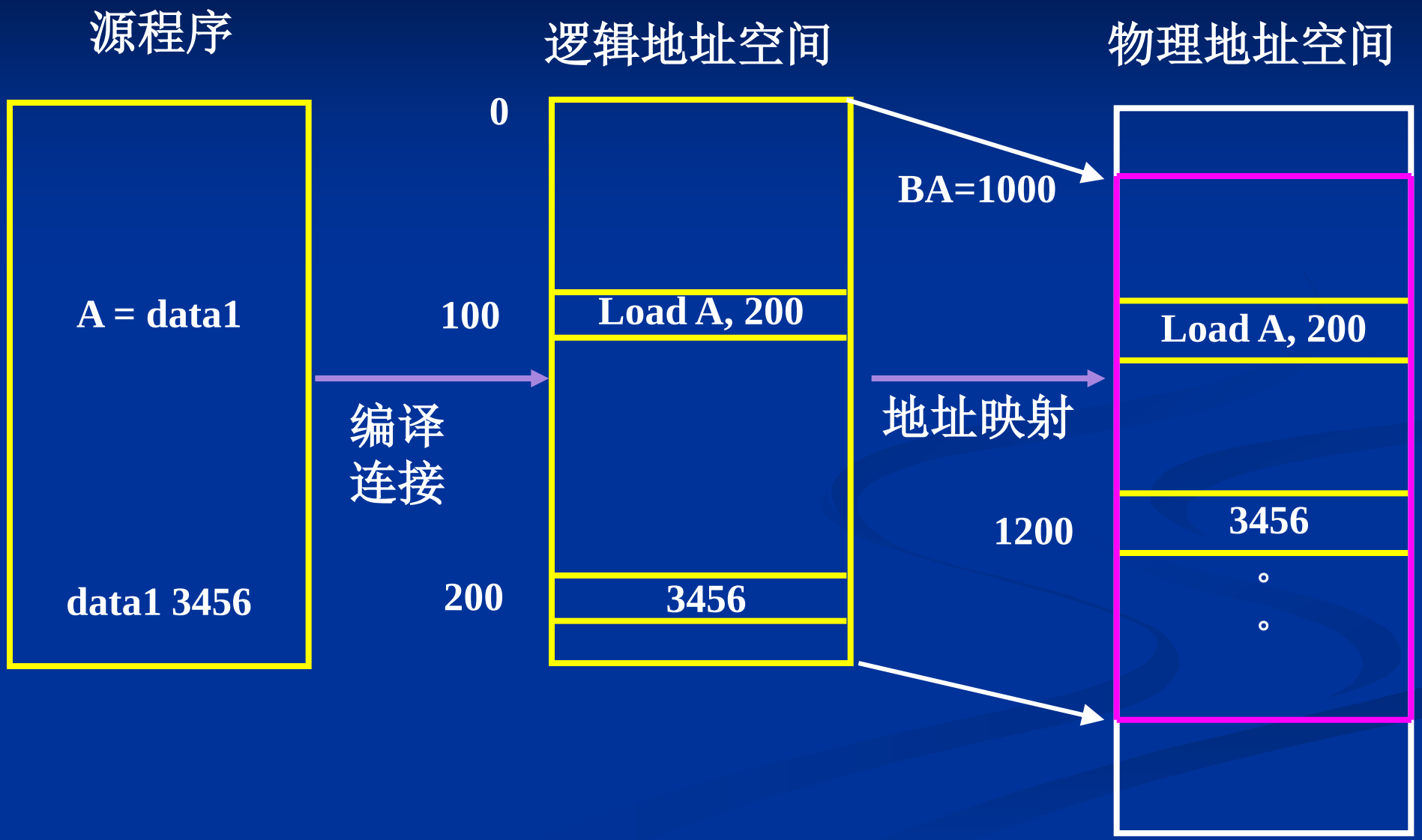
  当用户程序被装入内存时，一次性实现逻辑地址到物理地址的转换，以后不再转换

  一般在装入内存时由软件完成

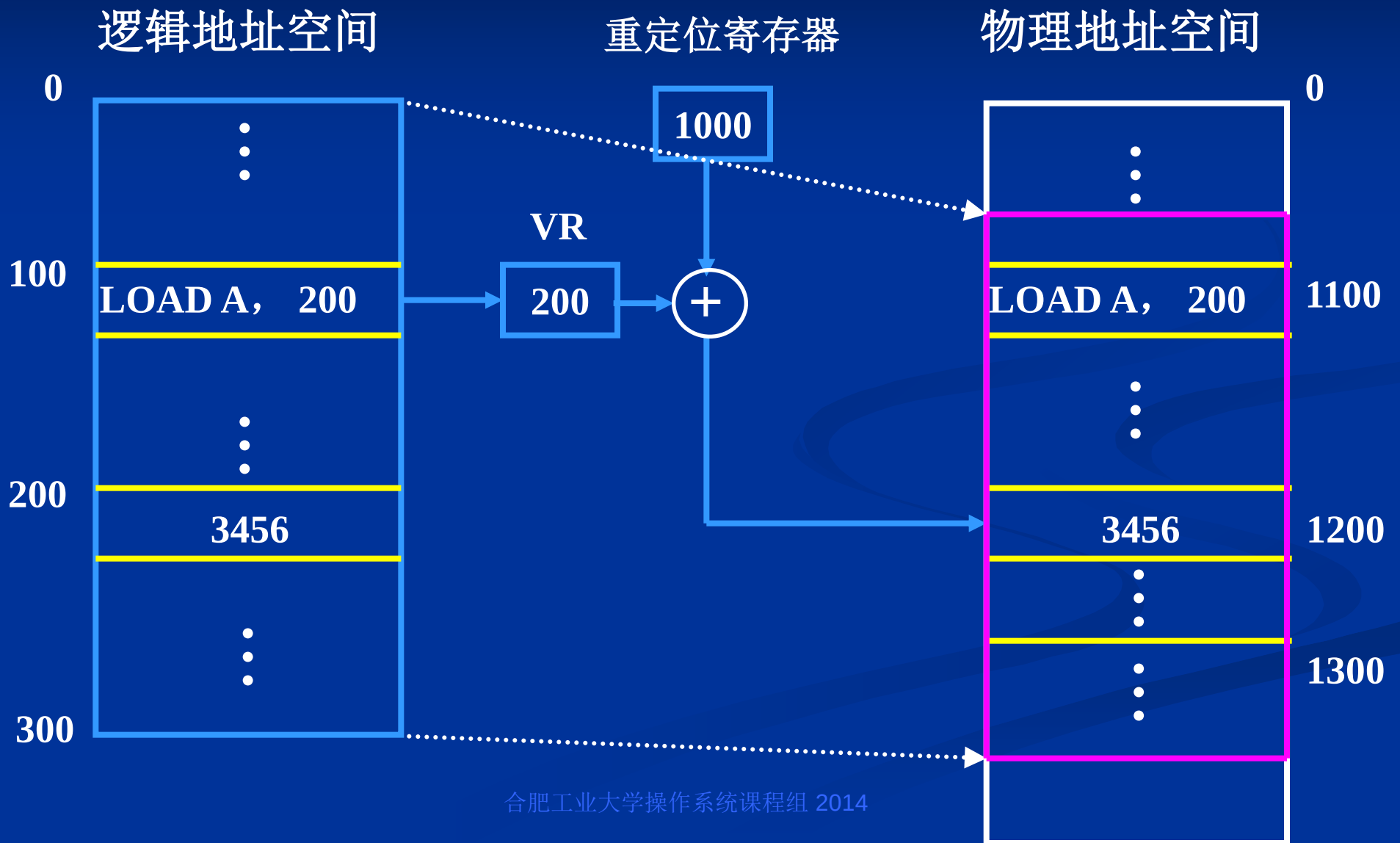
- 动态地址重定位

  在程序运行过程中要访问数据时再进行地址变换（即在逐条指令执行时完成地址映射。一般为了提高效率，此工作由硬件地址映射机制来完成。硬件支持，软硬件结合完成）

  硬件上需要一对寄存器的支持


源程序
逻辑地址空间
物理地址空间
A = data1
data1 3456
0
100
200
Load A, 200
3456
编译
连接
BA=1000
地址映射
1200
Load A, 200
3456

动态地址重定位


逻辑地址空间
重定位寄存器
物理地址空间
0
100
200
300
1000
VR
200
+
LOAD A， 200
3456
0
1100
1200
1300
LOAD A， 200
3456

4.1 程序的装入和链接

4.1.1 程序的装入

- 动态运行时装入装入方式

  编程时，使用逻辑地址，运行时使用物理地址； 运行时进行地址重定位；

  特点：

  灵活;支持多道程序设计技术；支持程序“搬家”

4.1.2 程序的链接

- 静态链接

  编程时，链接所有模块；

- 装入时动态链接

  装入时，链接所有模块；

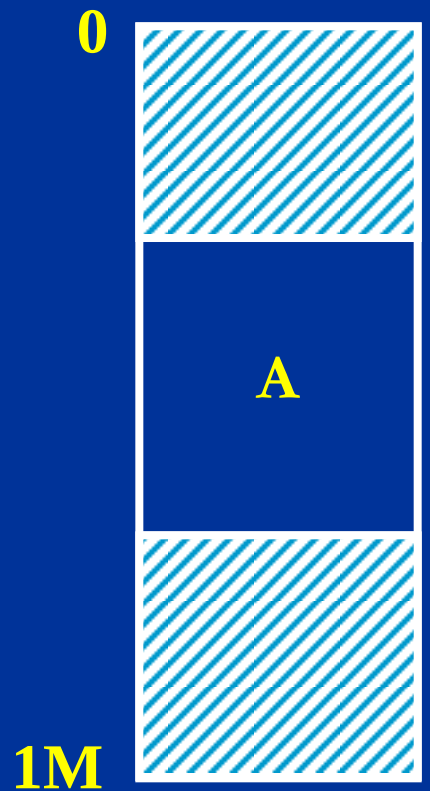
- 运行时动态链接

  运行时，根据需要链接模块；

4.2 连续分配方式

- 连续分配方式

  为程序分配一段连续存储空间。



- 离散分配方式

  为程序分配若干段连续的存储空间。



4.2.1 单一连续分配存储管理方案

(1) 应用背景

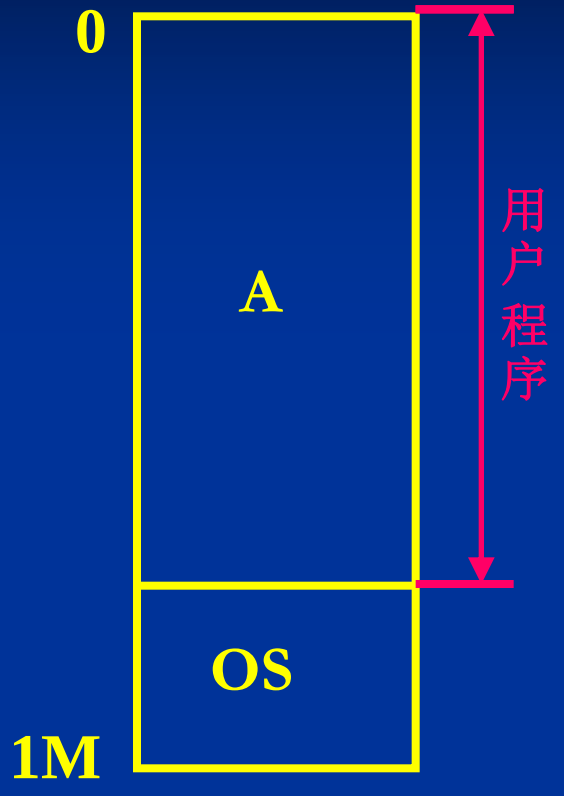
单用户系统。在一段时间内，只有一个进程在内存。

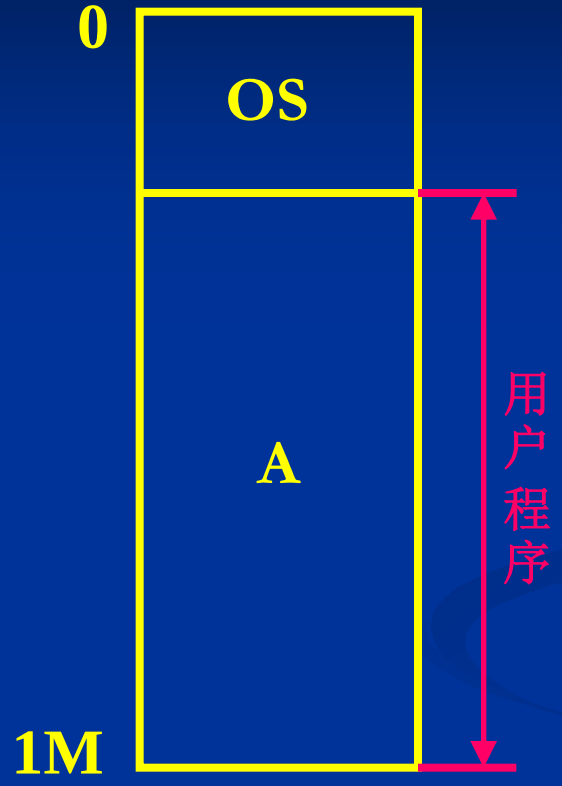
(2) 基本思想

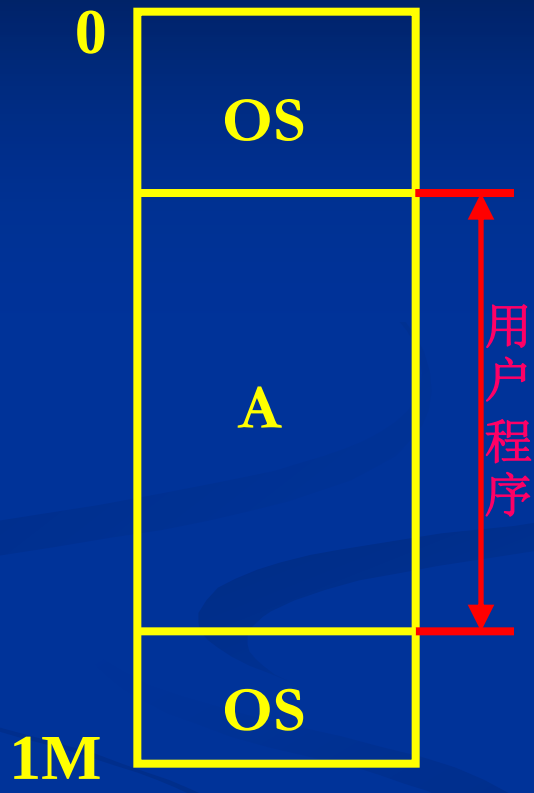
内存分为两个区域：一个供操作系统使用，一个供用户使用



三种典型内存布局：

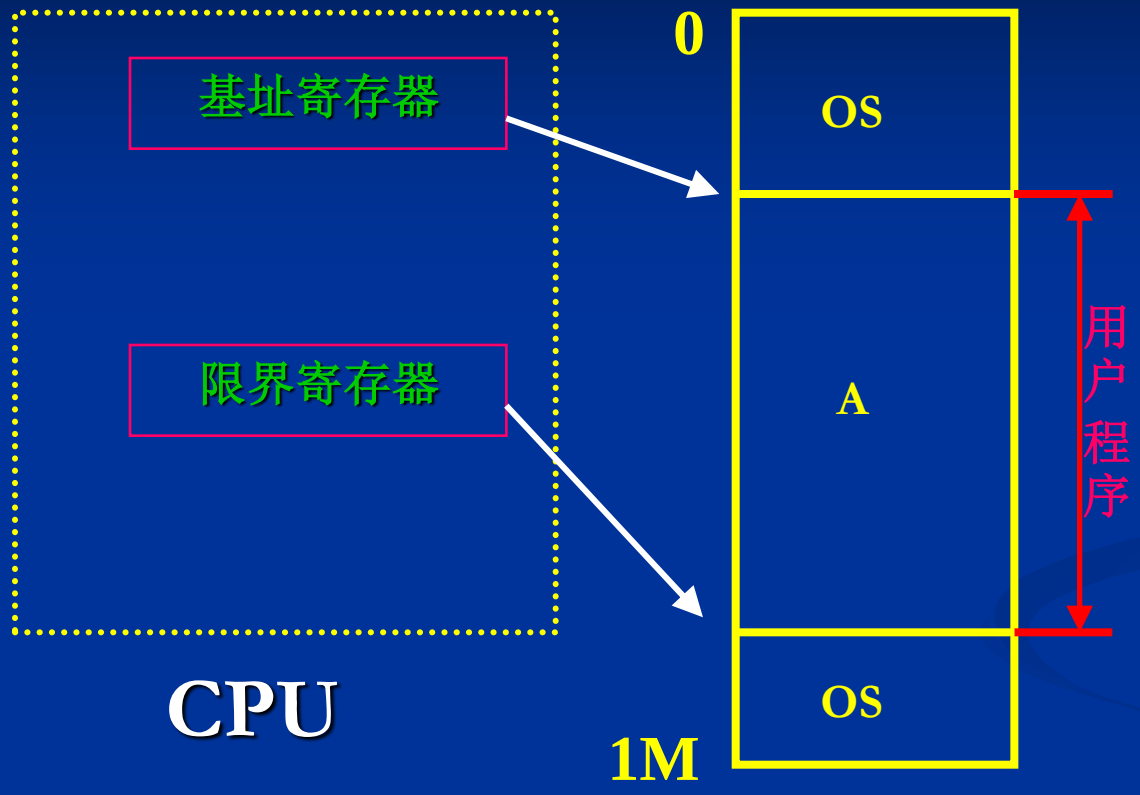






(3) 实现

设置基址寄存器、限界寄存器



(4) 特点

简单；适用于单任务OS；

4.2.2 固定分区分配存储管理方案

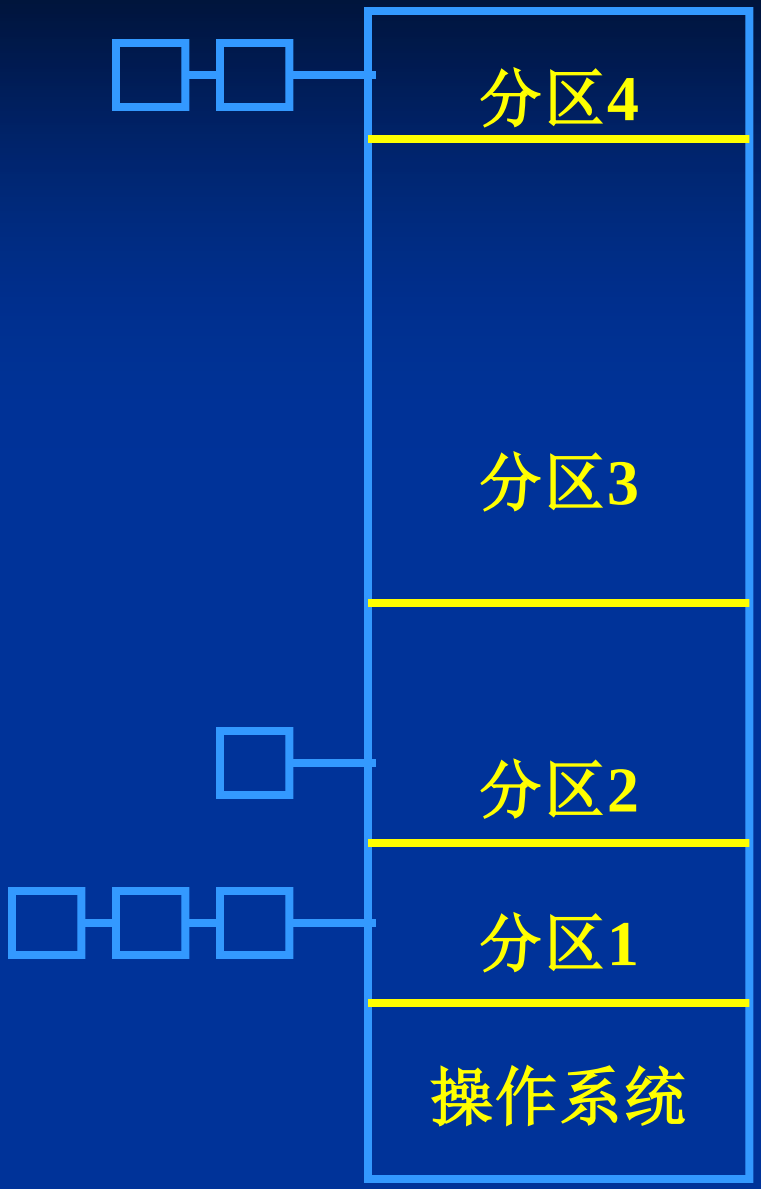
(1) 应用背景

多道程序设计系统。

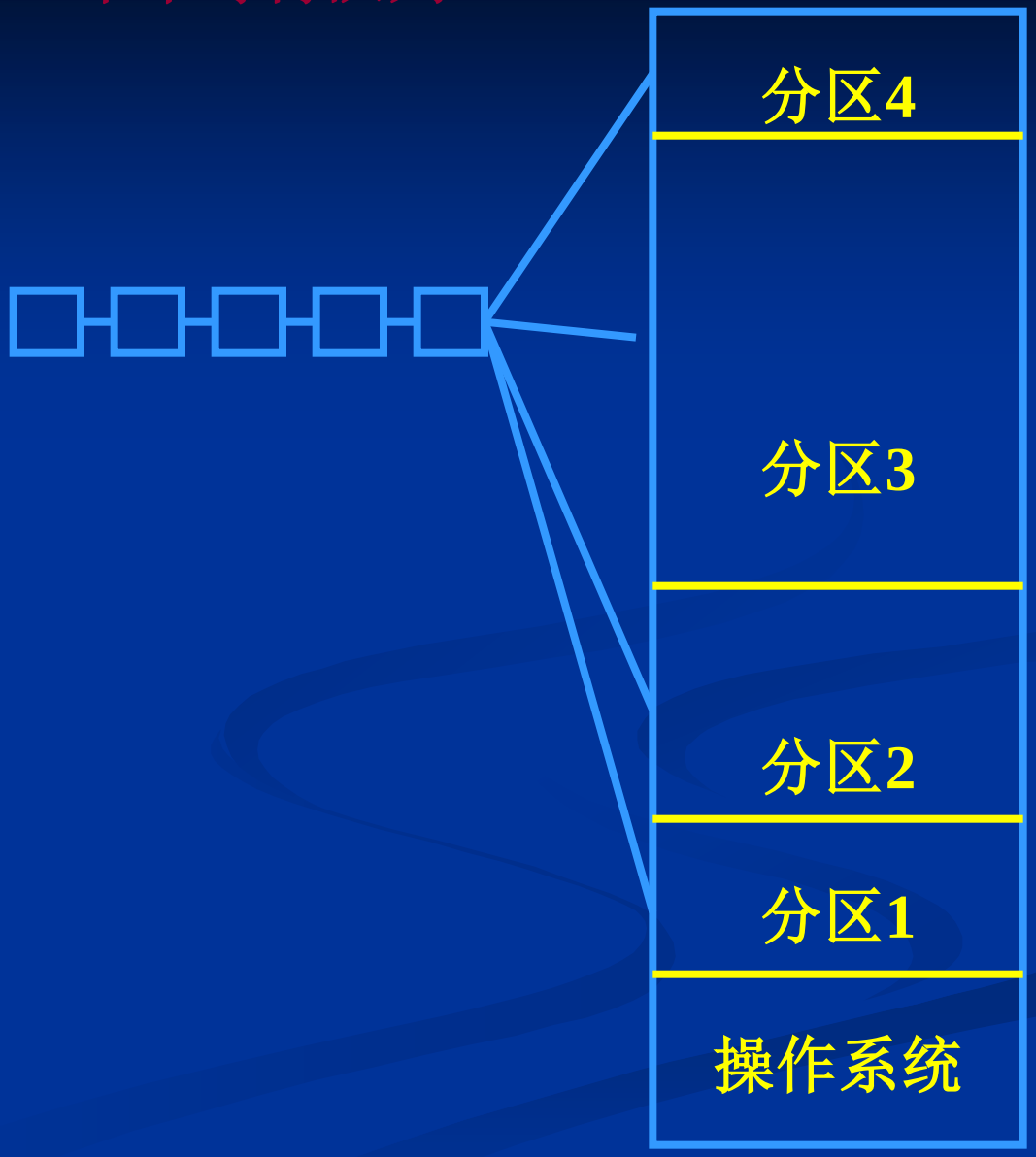
(2) 基本思想

- OS初始化阶段，将用户区划分为若干固定大小的内存区域（分区）；
- 每个分区可放一个程序运行；
- 一个分区内的程序执行结束，可调入另一个程序执行；
- 划分为几个分区，则允许几个程序同时在内存运行；
- 分区大小和位置可不同；

■多个等待队列



■单个等待队列

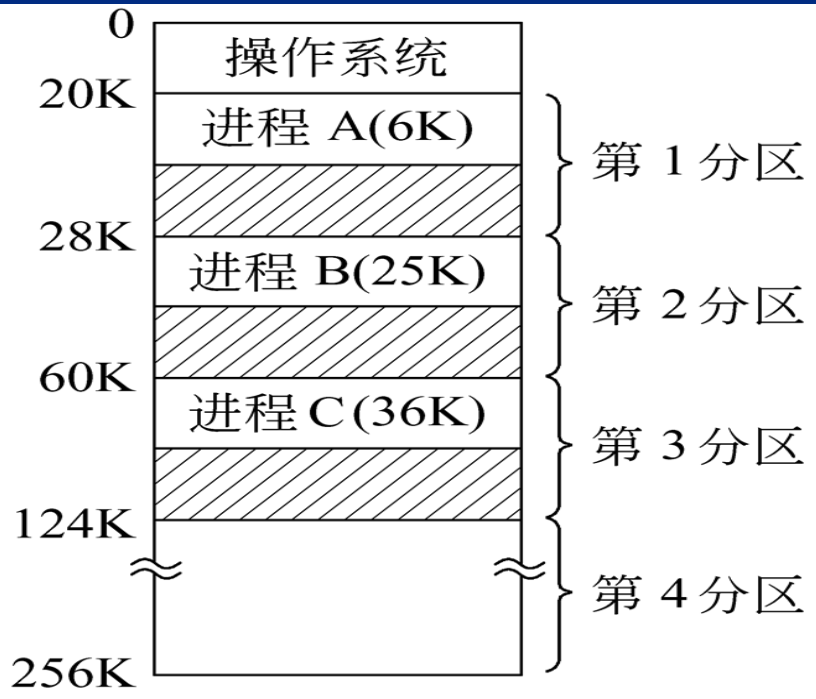


(3) 实现

- 数据结构：分区说明表

区号	分区长度	起始地址	状态
1	8K	20K	已分配
2	32K	28K	已分配
3	64K	60K	已分配
4	132K	124K	未分配

(a) 分区说明表



(b) 内存状态

- 算法：分区分配算法；分区回收算法

(4) 特点

- 最早的支持多道程序设计的存储管理方案；
- 简单，现在仍在嵌入式系统中使用；
- “内零头”严重；

内存的零头：

内零头：分配给进程，而进程未用到的内存部分；

外零头：未分配给进程，但因为太小而无进程能用；

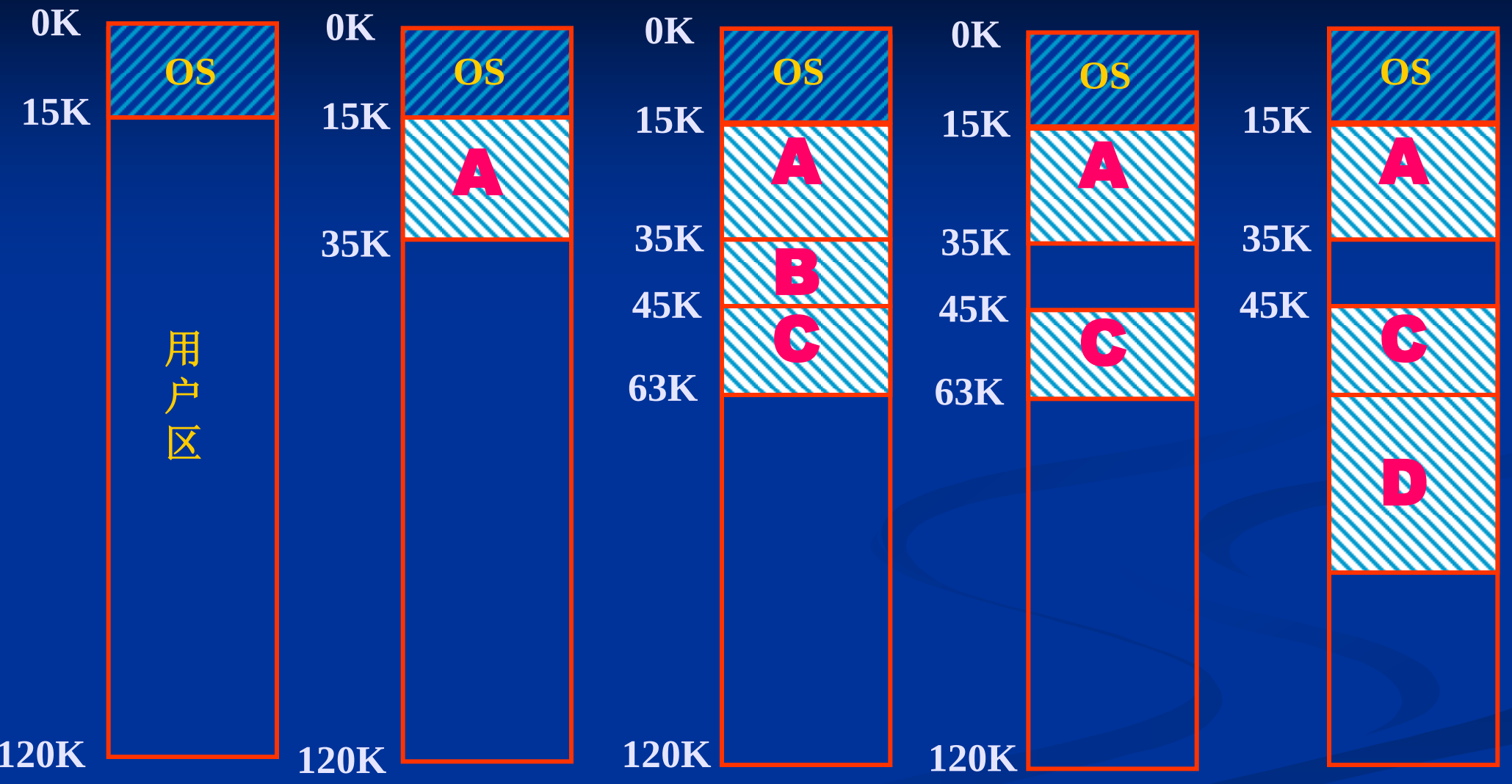
4.2.3 动态分区分配存储管理方案

(1) 应用背景

多道程序设计系统。

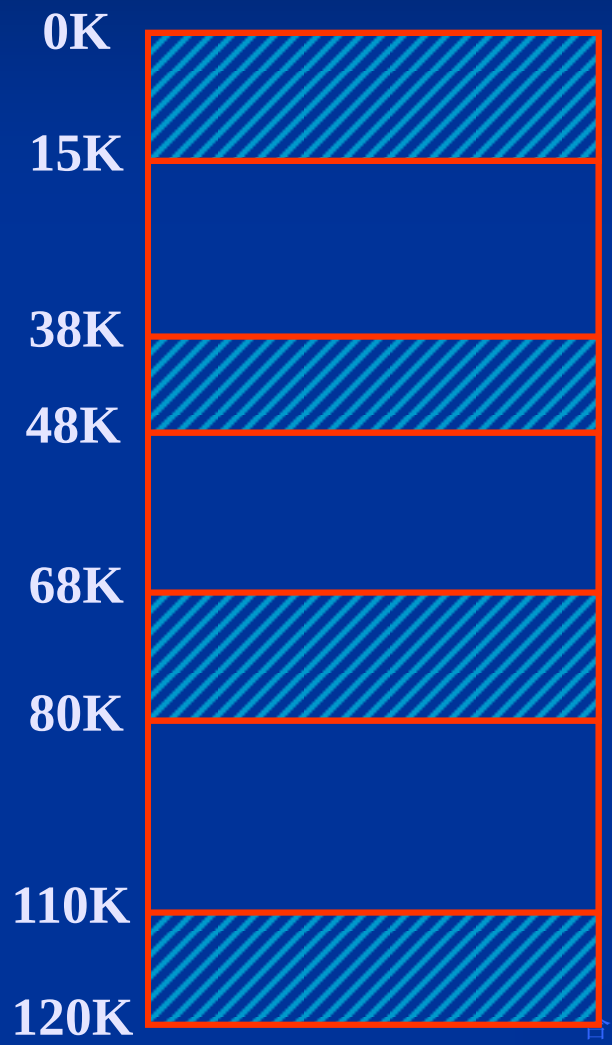
(2) 基本思想

- OS初始化阶段，将用户区划分为一个大的空白分区；
- 每个程序运行时，划分出一个大小合适的分区；
- 程序执行结束，其所在的分区撤消；


0K
OS
15K
用户区
120K
0K
OS
15K
A
35K
120K
0K
OS
15K
A
35K
B
45K
C
63K
120K
0K
OS
15K
A
35K
45K
C
63K
120K
OS
15K
A
35K
45K
C
D

(3) 实现

- 数据结构： (a)空闲分区表

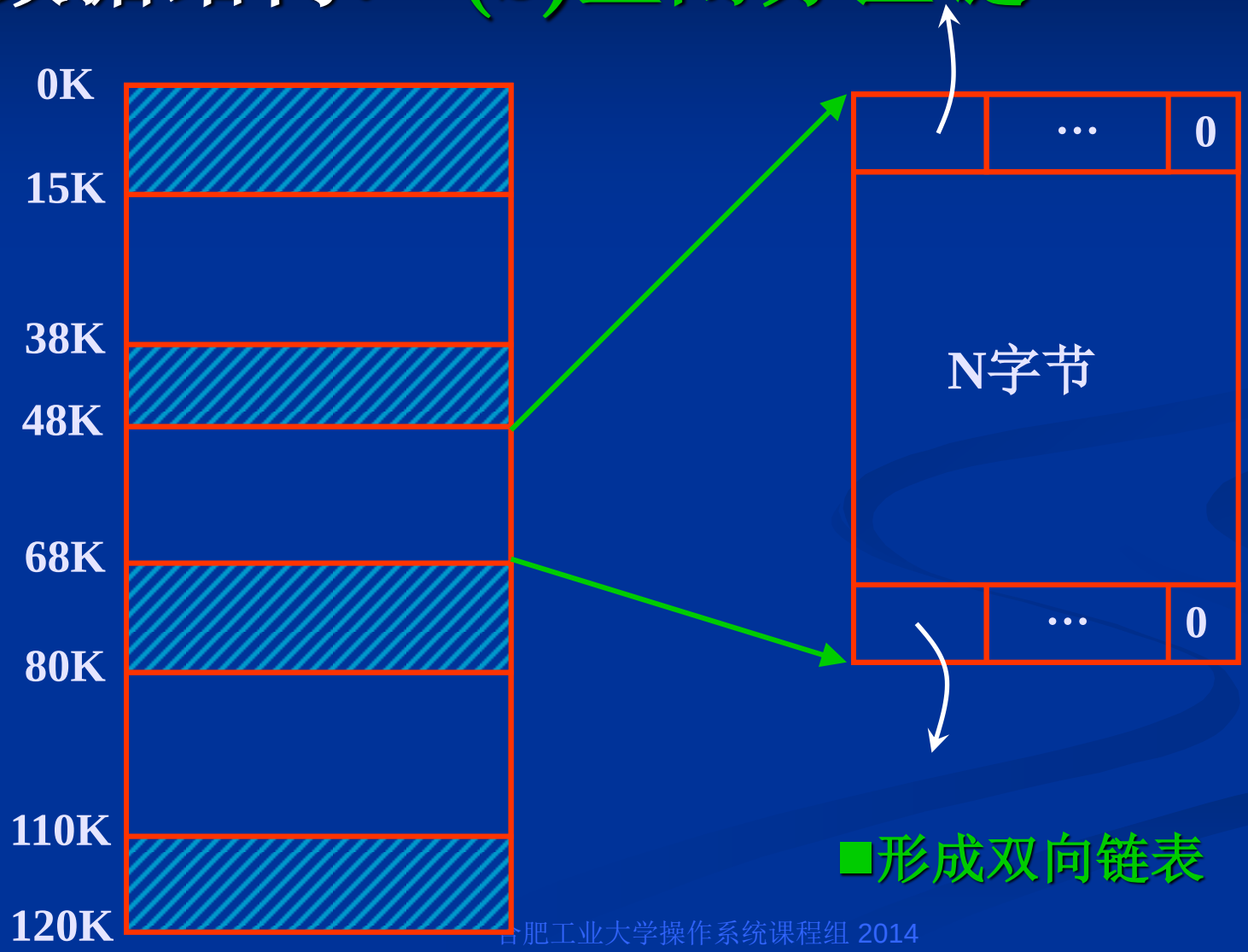


空闲分区表

始址	长度	标志
15K	23K	未分配
48K	20K	未分配
80K	30K	未分配
		空
		空

(3) 实现

- 数据结构： (b)空闲分区链



■形成双向链表

(3) 实现

(a) 分区分配算法

1) 首次适应算法

- 按地址递增顺序排列空闲分区链；
- 分配内存时，总是从低地址端开始扫描空闲区；
- 找到的第一个大小合适的分区，就分割分配；
- 否则失败。

特点：

- 高址端有大空闲区概率大；
- 低址端迅速被划分，碎片出现速度快；

2) 循环首次适应算法

- 按地址递增顺序排列空闲分区链；
- 设置当前指针；
- 分配内存时，从指针所指位置开始扫描空闲区；
- 找到的第一个大小合适的分区，就分割分配；
- 否则失败。

本质：

CLOCK算法；

特点：

- 碎片分布均匀；
- 高址端有大空闲区概率小；

3) 最佳适应算法

- 按大小递增顺序排列空闲分区链；
- 分配内存时，从链首开始扫描空闲区；
- 找到的第一个大小合适的分区，就分割分配；
- 否则失败。

特点：

- 碎片迅速出现；
- “最佳”匹配；

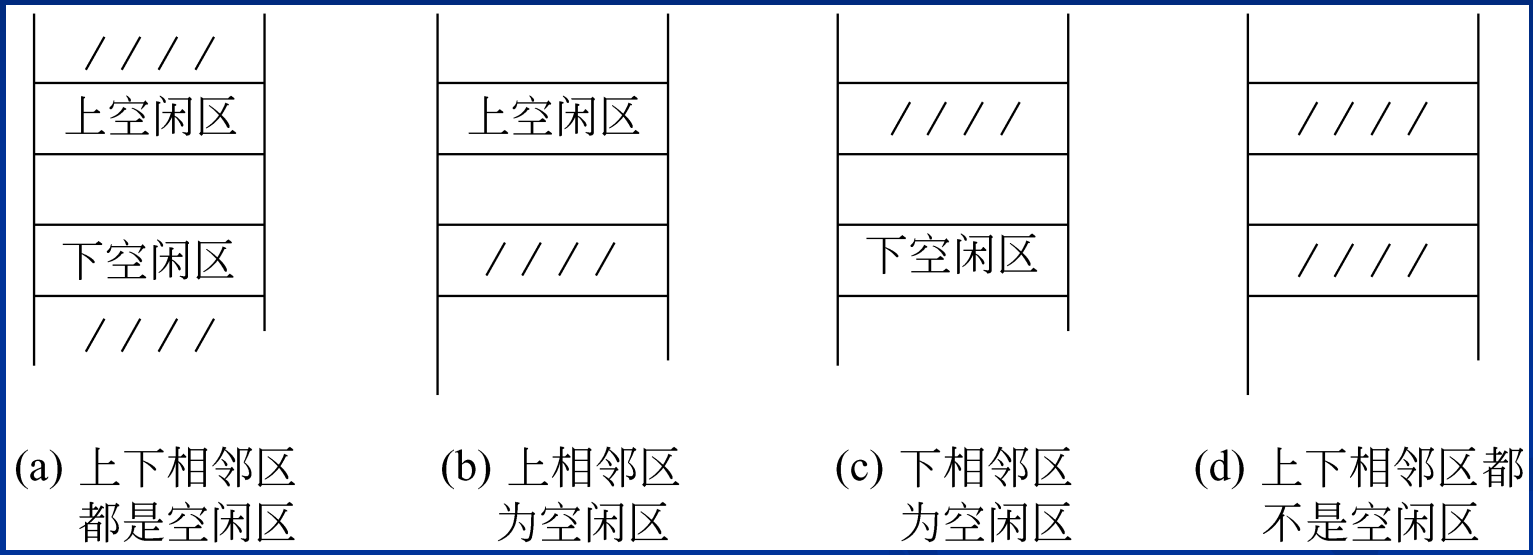
4) 最坏适应算法

- 按大小递减顺序排列空闲分区链；
- 分配内存时，总是分配链首空闲区；
- 否则失败。

特点：

- 碎片出现最慢（分割后剩下的分区是最大的）；
- 无大的空闲区保留；

(b) 分区回收算法



(a) 上下相邻区都是空闲区　(b) 上相邻区为空闲区　(c) 下相邻区为空闲区　(d) 上下相邻区都不是空闲区

（a）和上下一起合并　　（c）和下一起合并

（b）和上一起合并　　（d）自己成为一个新区

(4) 特点

- “碎片”问题

  经过一段时间的分配回收后，内存中存在很多很小的空闲块。它们每一个都很小，不足以满足分配要求；但其总和满足分配要求。这些空闲块被称为碎片造成存储资源的浪费

4.2.3 可重定位分区分配存储管理方案

(1) 应用背景

多道程序设计系统。

(2) 基本思想

- 采用动态分区分配方式;
- 引入“紧凑”机制，合并小的碎片空闲区;
- 使用动态地址重定位;



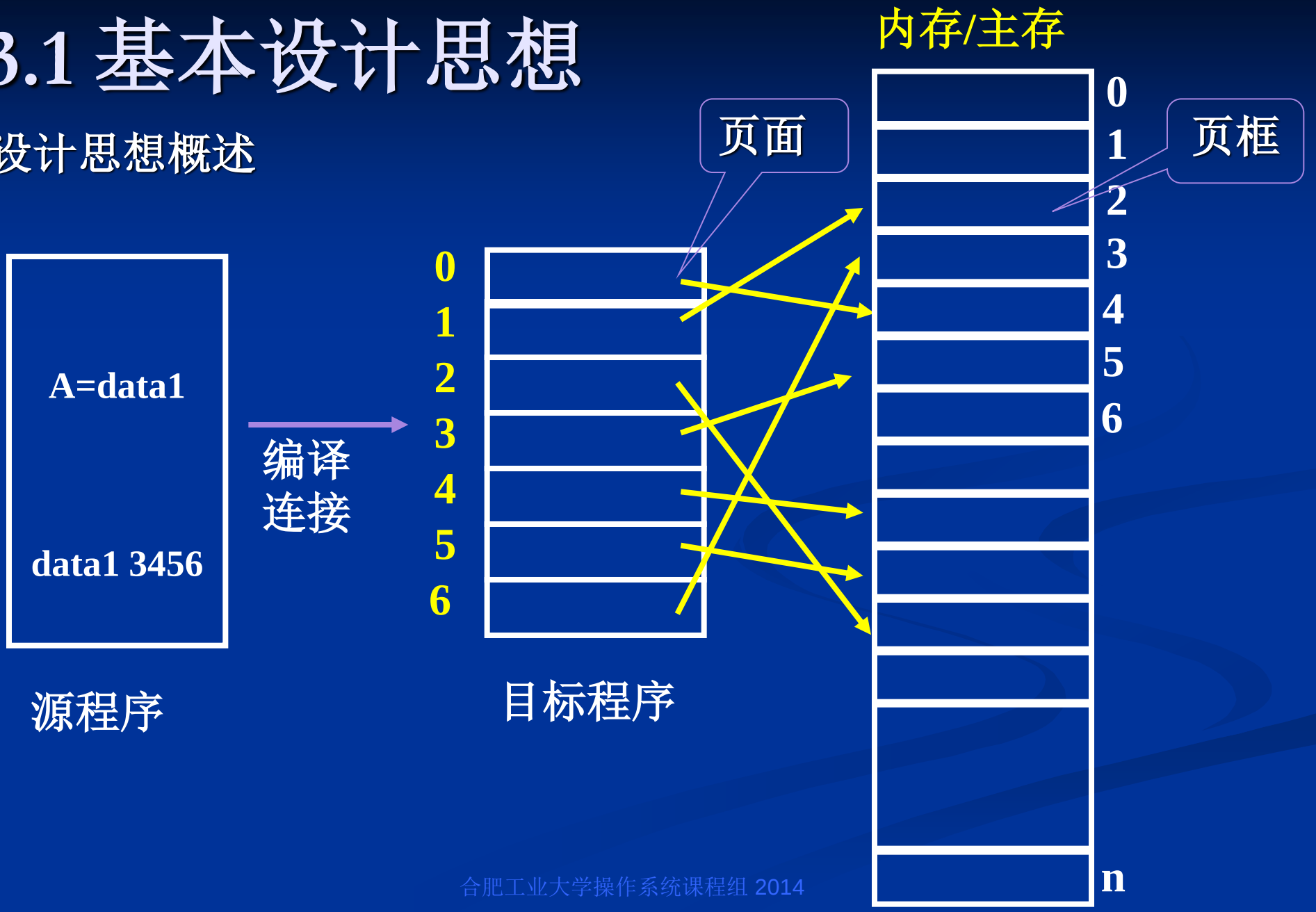
- 算法设计问题：如何紧凑，才能保证算法效率最高？

4.3 基本分页存储管理方式

- 分区式分配存在的问题:
  碎片（内外“零头”）问题难以解决。

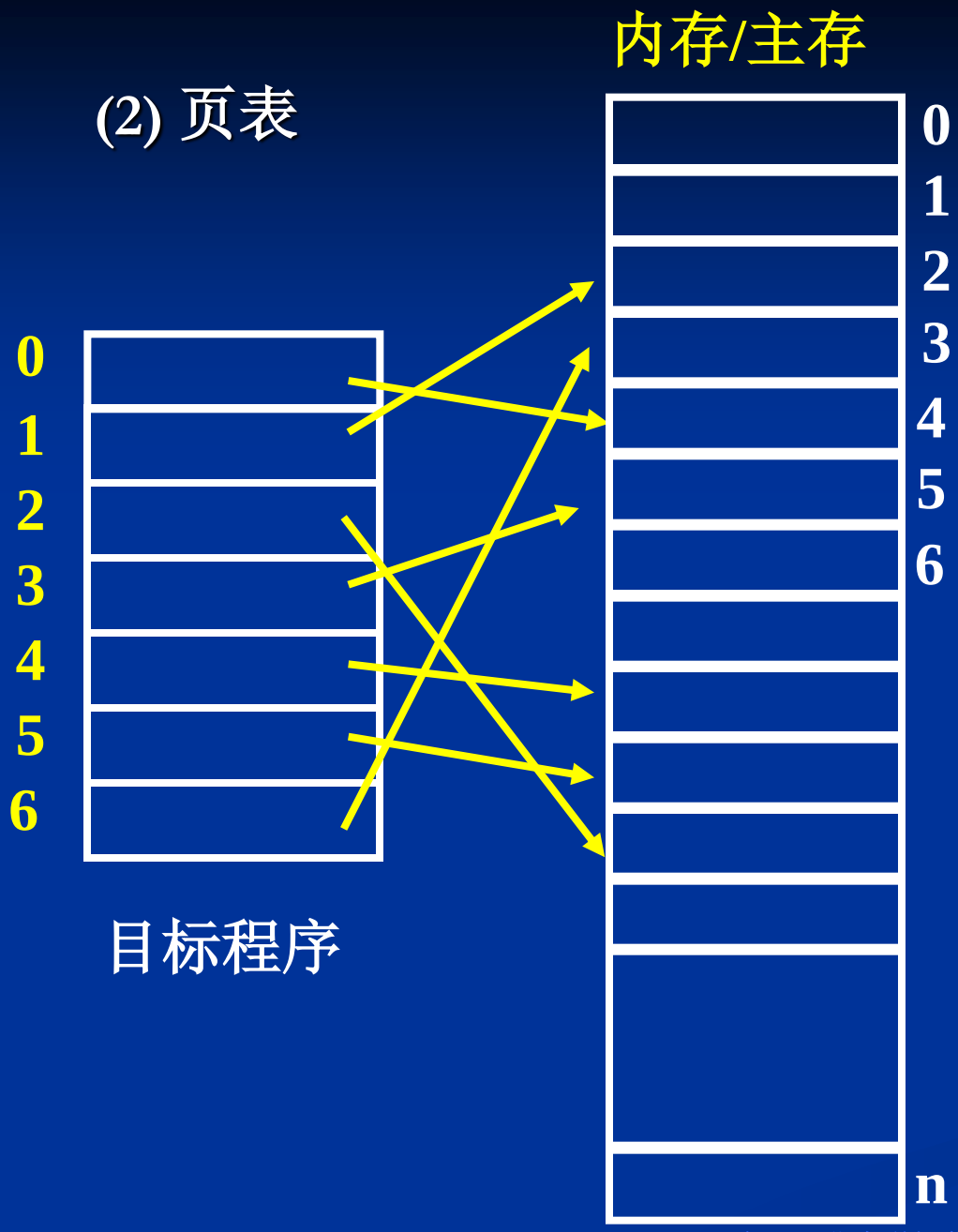
4.3.1 基本设计思想

(1) 设计思想概述



- 要解决的问题：
  - 页面/页框尺寸；
  - 逻辑地址结构；
  - 记录每个页面储存在哪个页框；
  - 程序如何正确运行（地址重定位）？
  - 指令跨页时的执行问题；

(2) 页表



页表:

页号	页框号	存取控制
0	4	X
1	2	X
2	10	X
3	5	X
4	8	RW
5	9	X
6	3	RW

- 页表存储在内存;
- 页表起始地址保存在PCB;

(3) 页面尺寸

页面太小：

内存利用率（内零头）高，页面数量增加，占用内存；

页面太大：

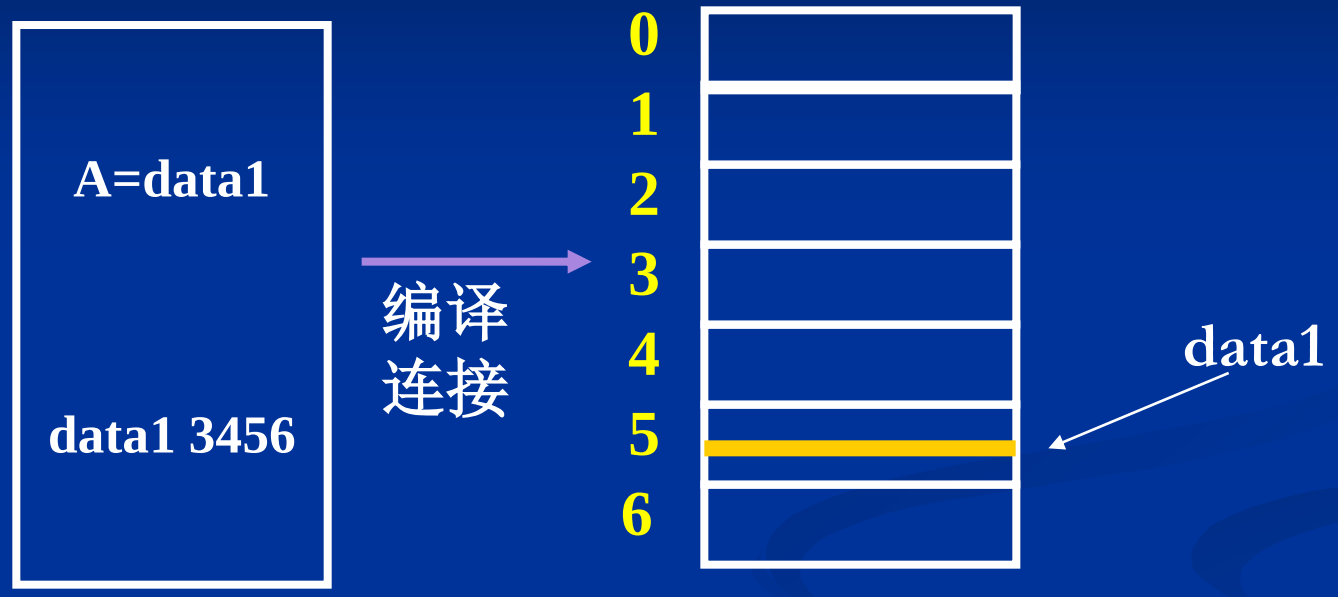
内存利用率（内零头）低，页面数量减少，节省内存；

正常尺寸：

2的整数次方，512-8K

(4) 地址结构

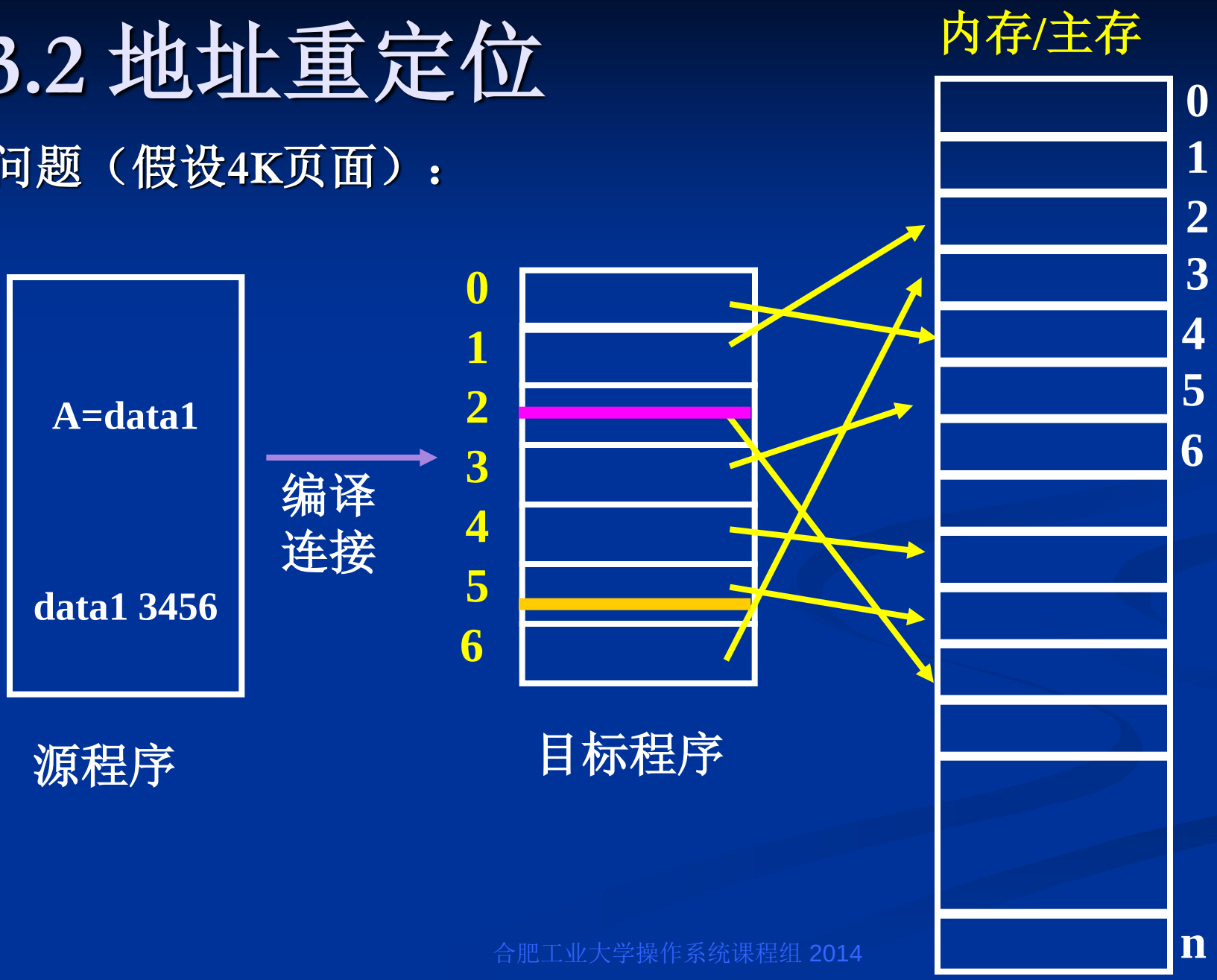
包括：页号和页内地址2部分。





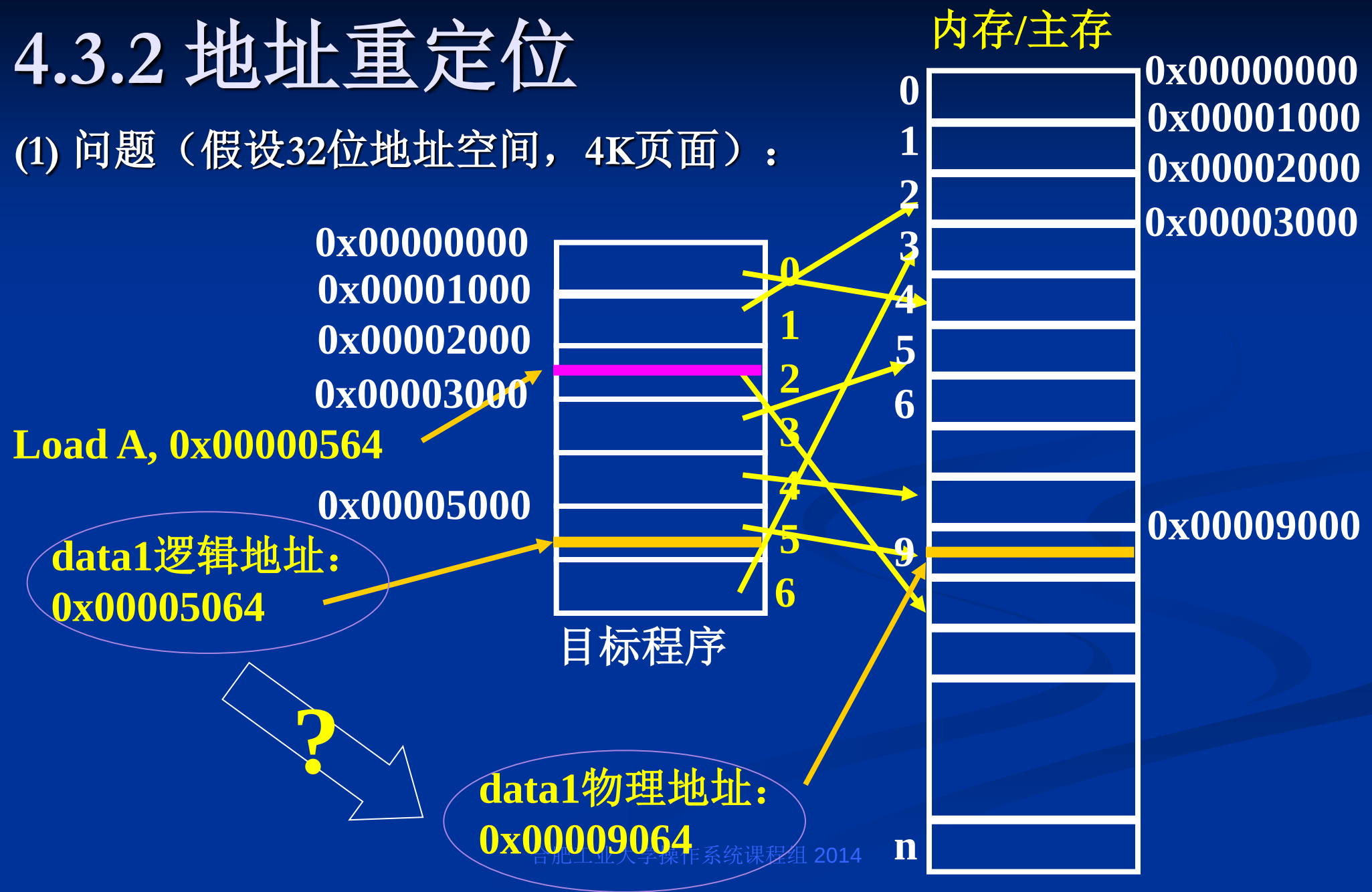
4.3.2 地址重定位

(1) 问题（假设4K页面）：

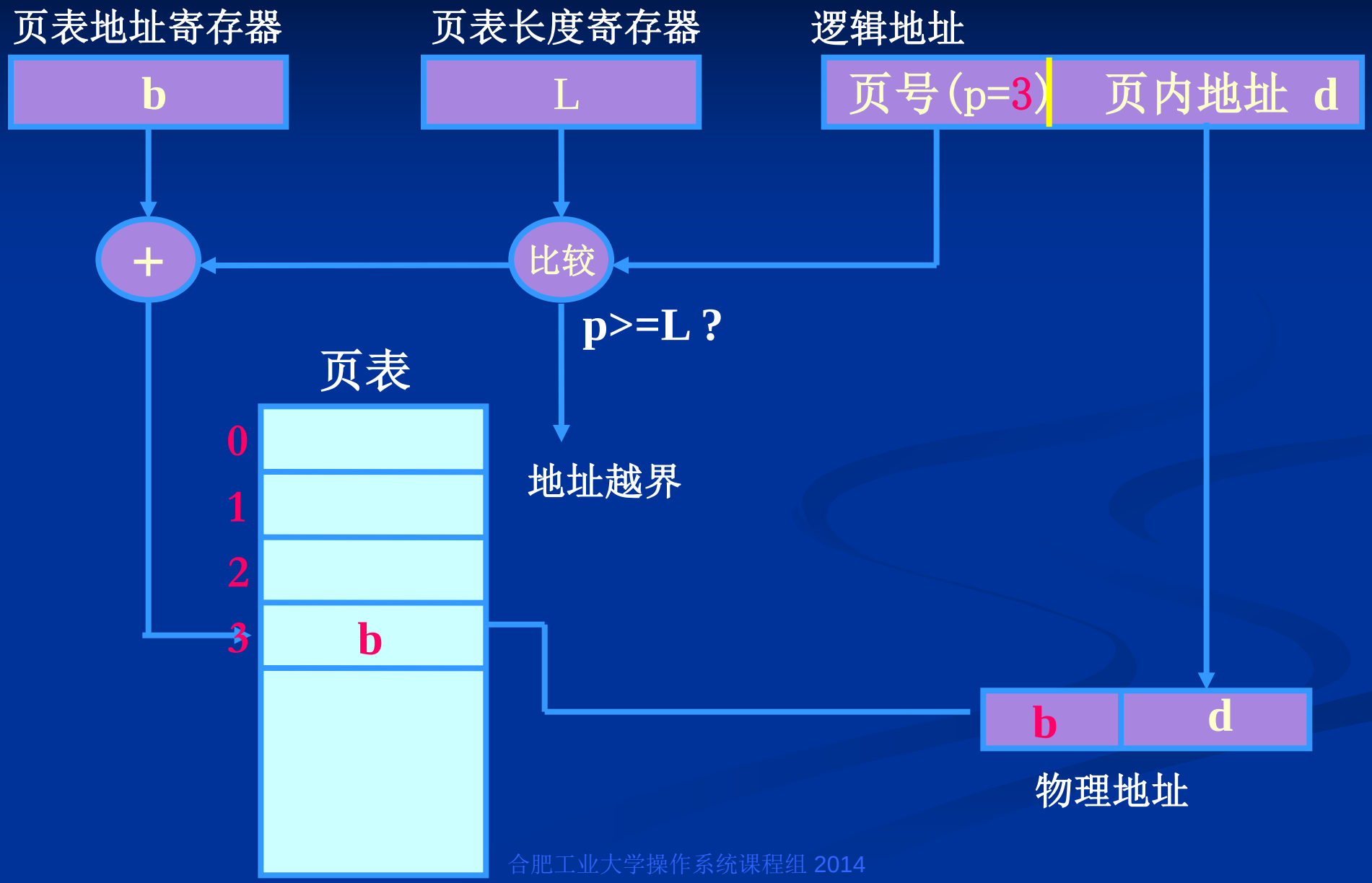


4.3.2 地址重定位

(1) 问题（假设32位地址空间，4K页面）：



(2) 基本地址变换机构/地址映射机制



(2) 基本地址变换机构/地址映射机制：计算示例

假设32位地址空间，4K页面



data1逻辑地址：0x00005064

9

data1物理地址：0x00009064

页表：

页号	页框号	存取控制
0	4	X
1	2	X
2	10	X
3	5	X
4	8	RW
5	9	X
6	3	RW

- **页表的作用**：分页系统的核心数据结构
  - 记录程序各页面所在的页框位置；
  - 支持进行地址重定位；
  - 实现页面访问控制；
  - 存储保护：限制程序在操作系统指定的内存区域内运行。

- 性能问题：

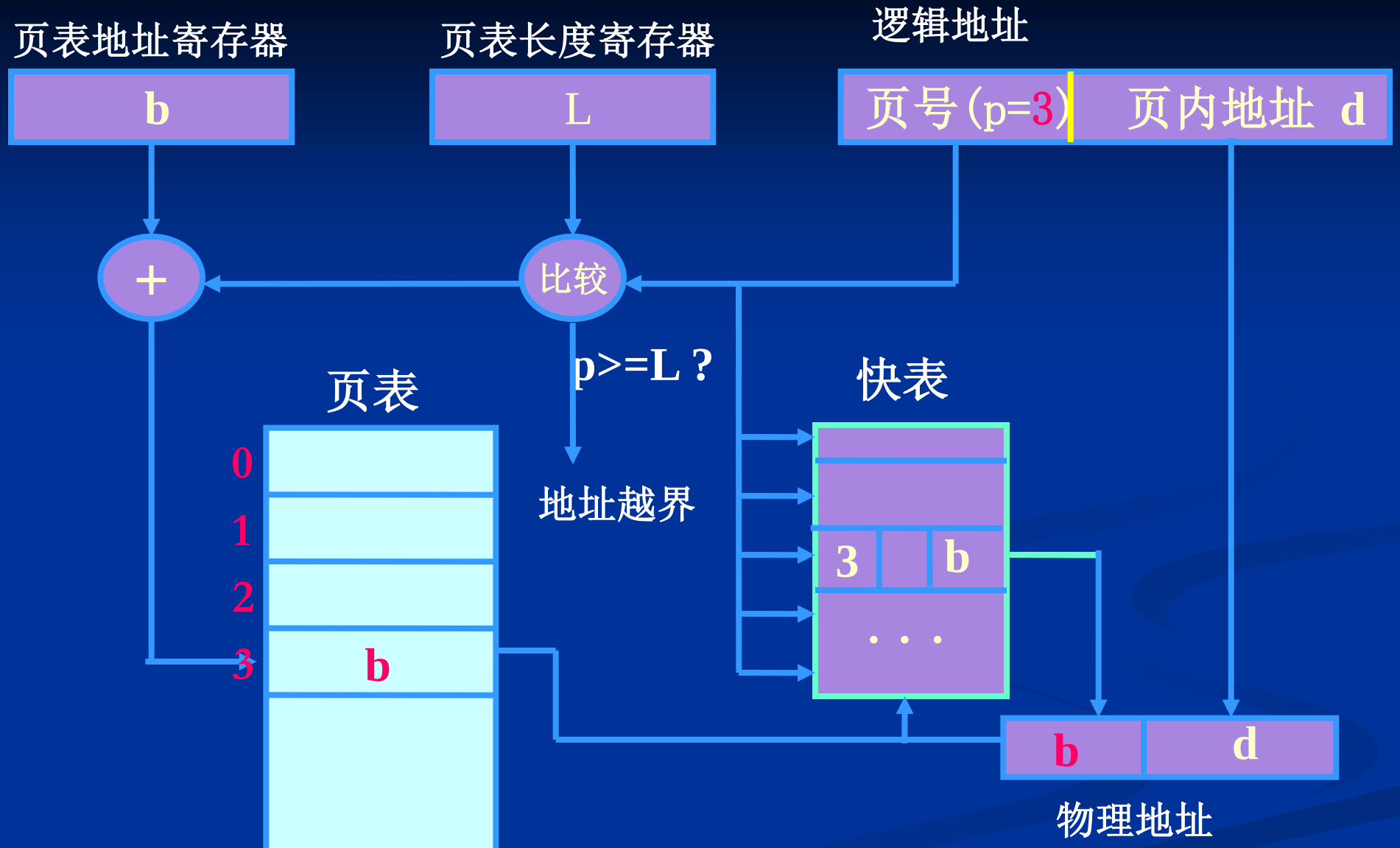
  访问1次内存变量，涉及2次地址访问：页表+变量

- 解决办法：快表
  - 设置在CPU内部；
  - 具有并行查找能力；
  - 暂存当前正在使用的页表项；
  - 尺寸：16-512 。可达到90%以上命中率
  - 别名：

    联想存储器（相联存储器）（associative memory）

    Intel术语：TLB(Translation lookaside buffers）

(3) 具有快表的地址变换机构



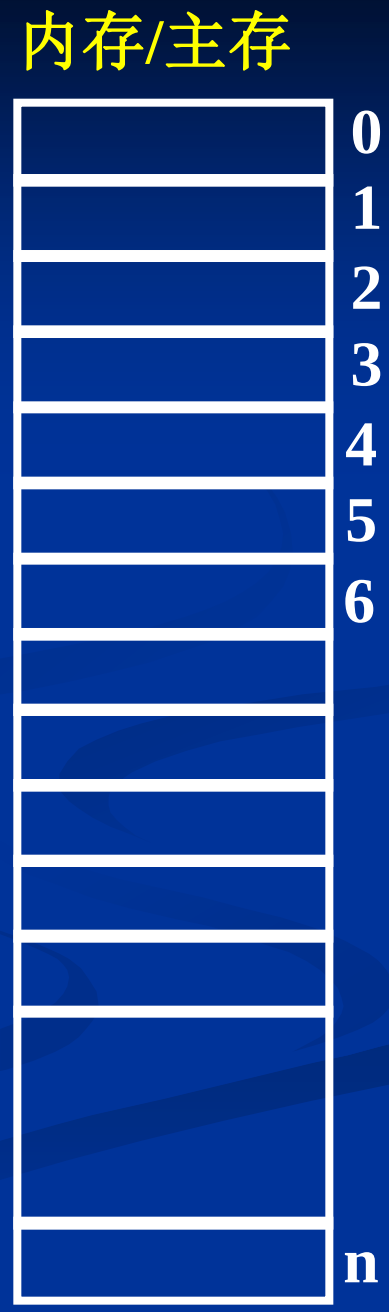
4.3.3 两级和多级页表

(1) 页表的存储问题（假设4K页面,4字节页表项）：

页表:

页号	页框号	存取控制
0	4	X
1	2	X
2	10	X
。 。	。 。 。	。 。 。
230	9879	RW
。 。	。 。 。	。 。 。
505	3	RW

占用1个页框



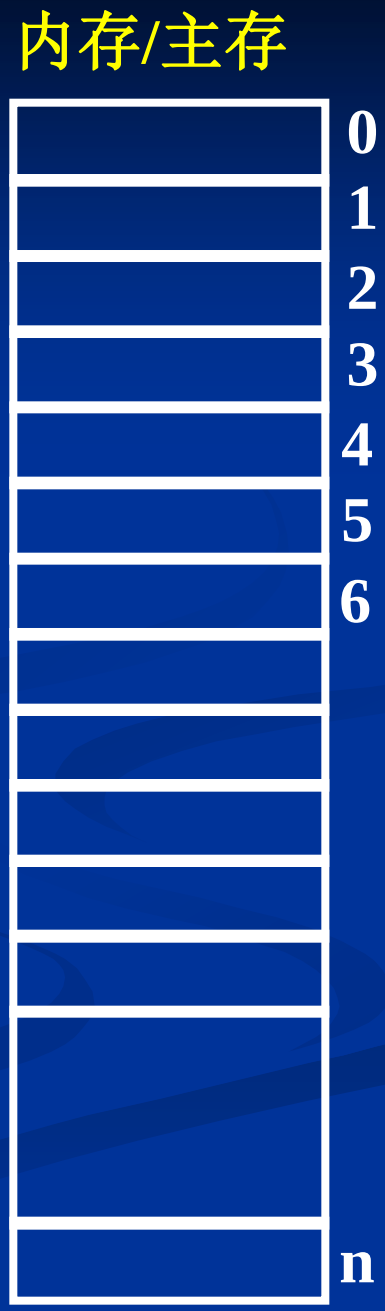
4.3.3 两级和多级页表

(1) 页表的存储问题（假设4K页面,4字节页表项）：

页表:

页号	页框号	存取控制
0	4	X
1	2	X
2	10	X
。。	。。。	。。。
1023	9879	RW
。。	。。。	。。。
5060	3	RW

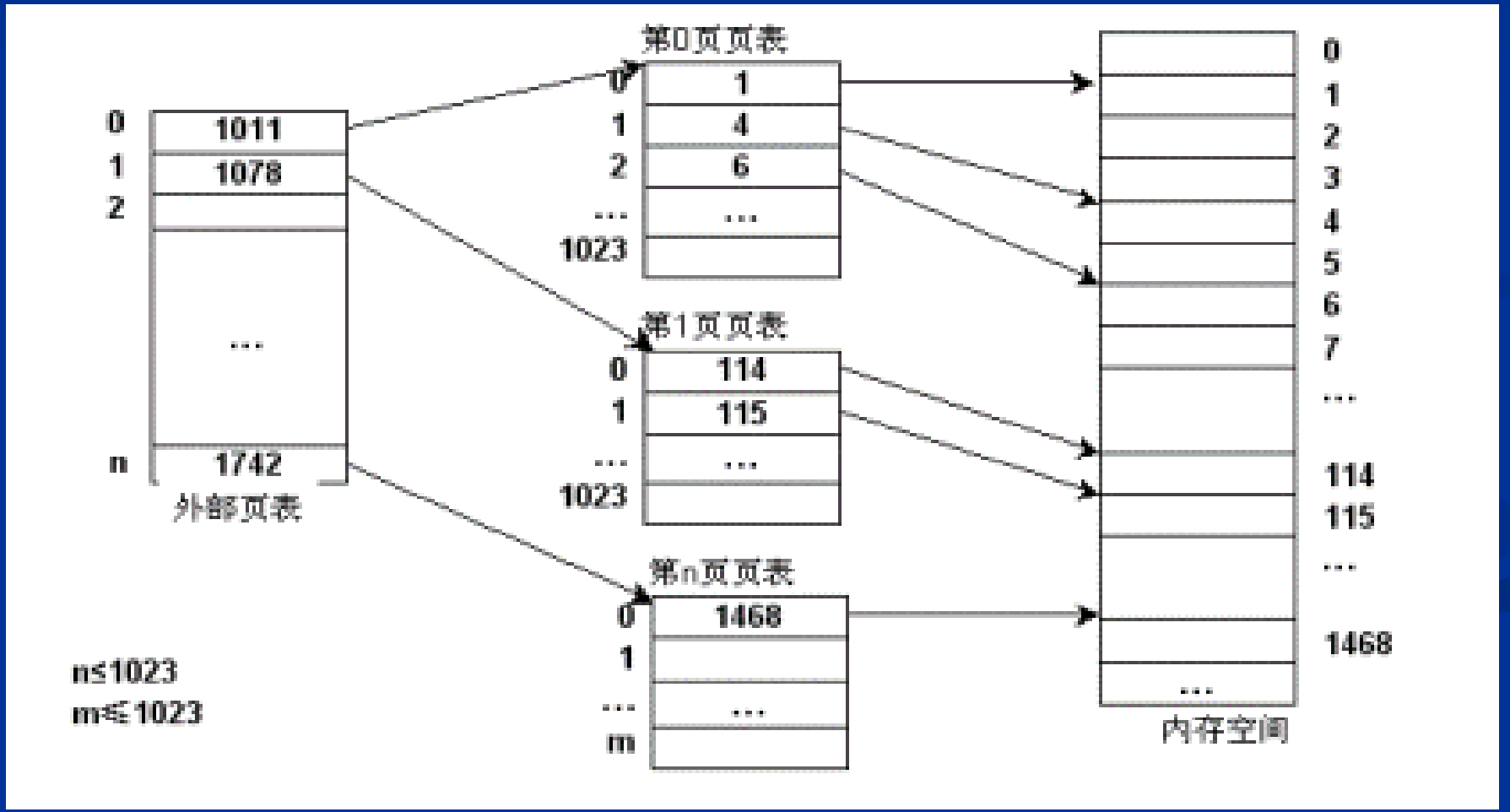
占用多个页框



4.3.3 两级和多级页表

(2) 两级页表

为页表再建立一张页表，以记录页表所占据的页框信息，从而形成两级页表。



4.3.3 两级和多级页表

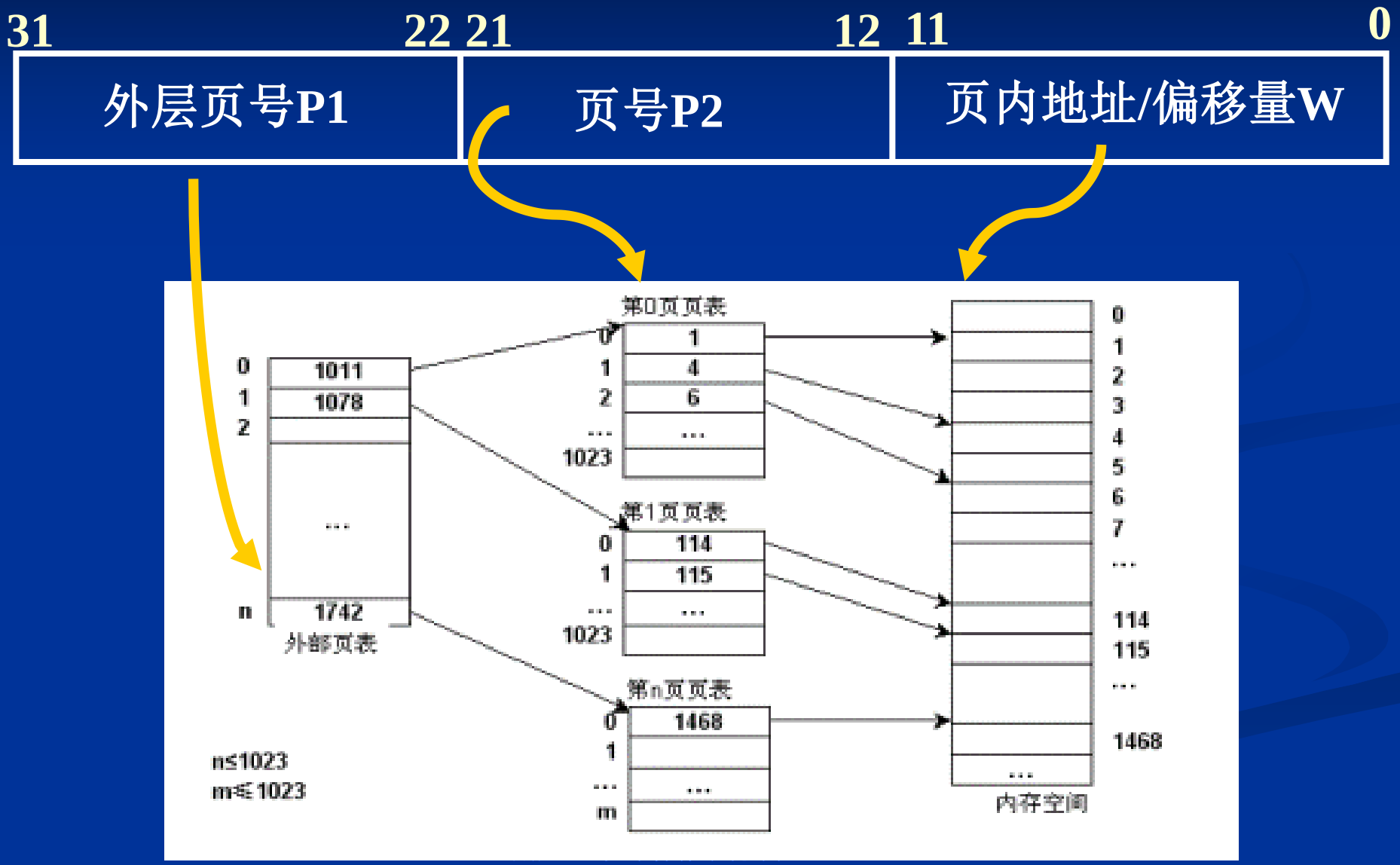
(2) 两级页表

- 两级页表地址结构：



(2) 两级页表

- 两级页表地址重新定位



4.3.3 两级和多级页表

(3) 多级页表

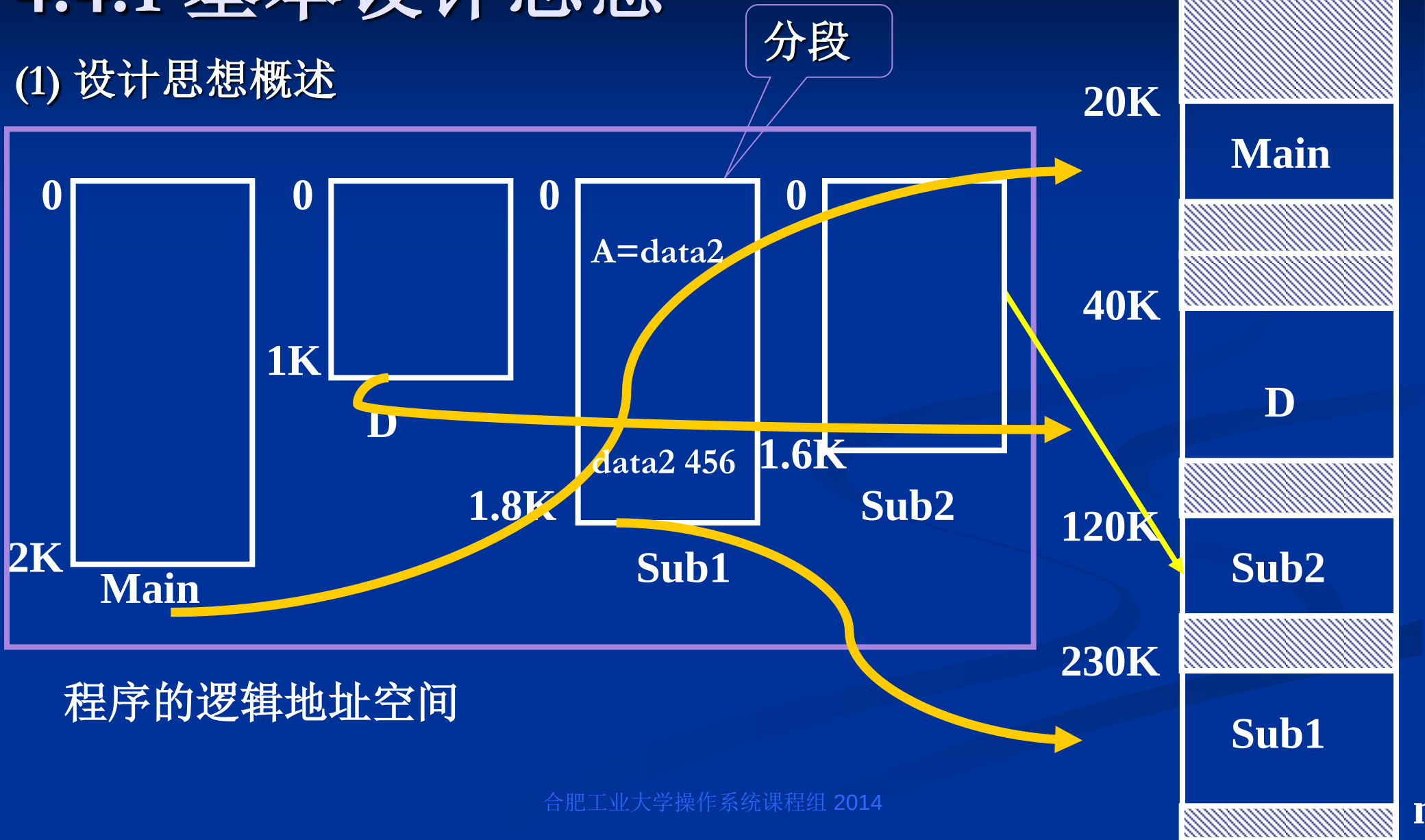
页表的2级索引推广到多级索引。

4.4 基本分段存储管理方式

- 分页系统存在的问题:
  - 信息共享和保护困难;
  - 编程不便;

4.4.1 基本设计思想

(1) 设计思想概述



程序的逻辑地址空间

4.4.1 基本设计思想

- 要解决的问题：
  - 分段划分；
  - 逻辑地址结构；
  - 记录每个分段储在内存位置；
  - 程序如何正确运行（地址重定位）？
  - 指令跨分段时的执行问题?

4.4.1 基本设计思想

(2) 分段

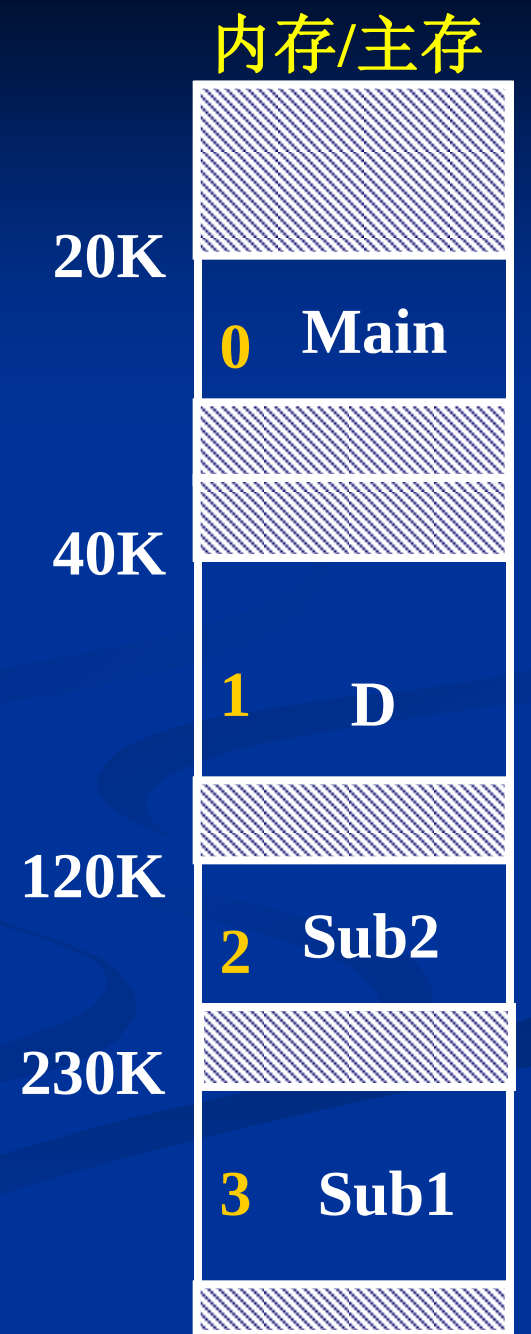
- 分段是一段有意义的信息集合；
- 分段的划分由程序员完成；
- 分段的长度不定；
- 指令不存在跨分段情况；

(3) 段表

段表:

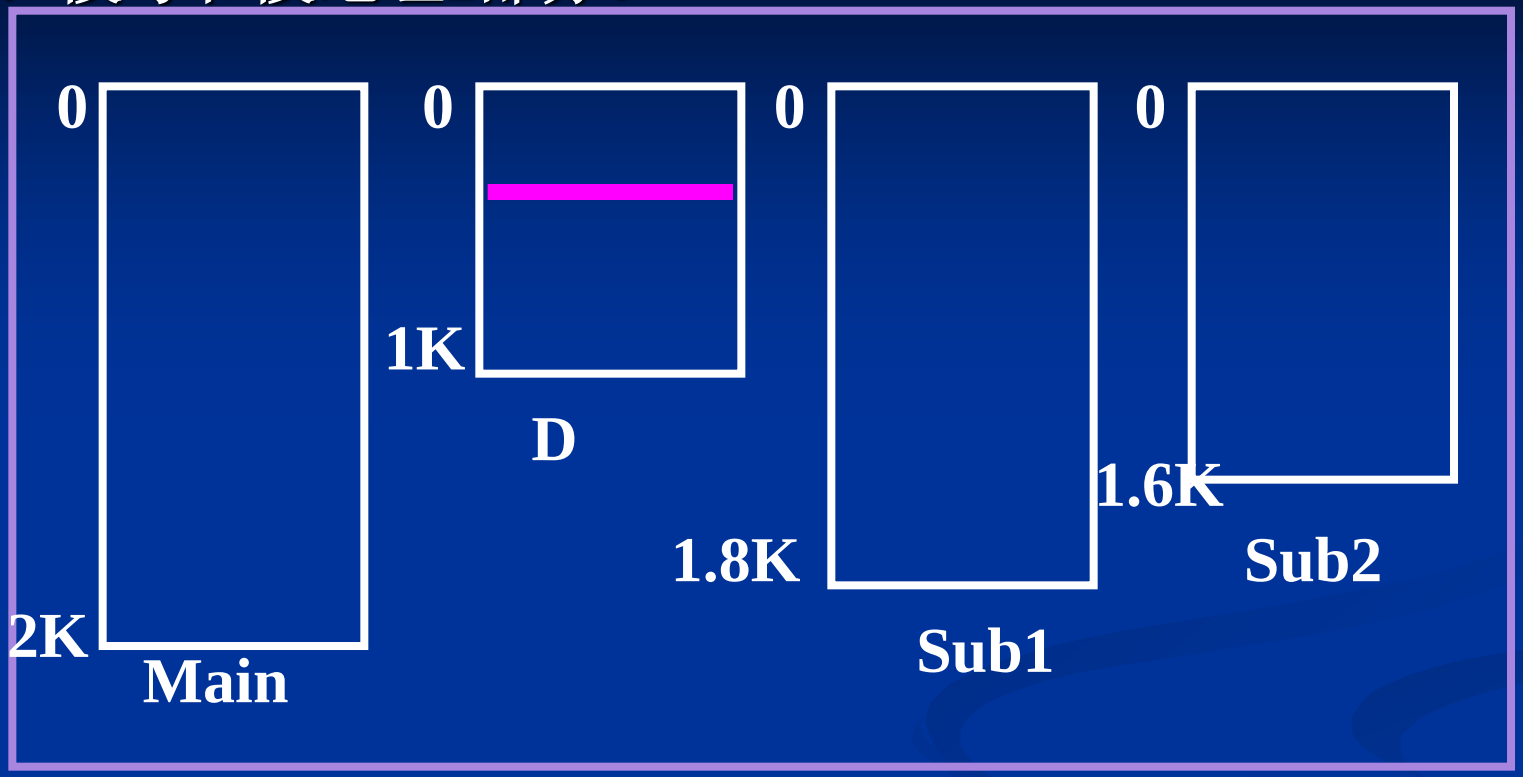
段号	段基址	段长	存取控制
0	20K	2K	X
1	40K	1K	RW
2	120K	1.6K	X
3	230K	1.8K	X

- 段表存储在内存;
- 段表起始地址保存在PCB;



(4) 地址结构

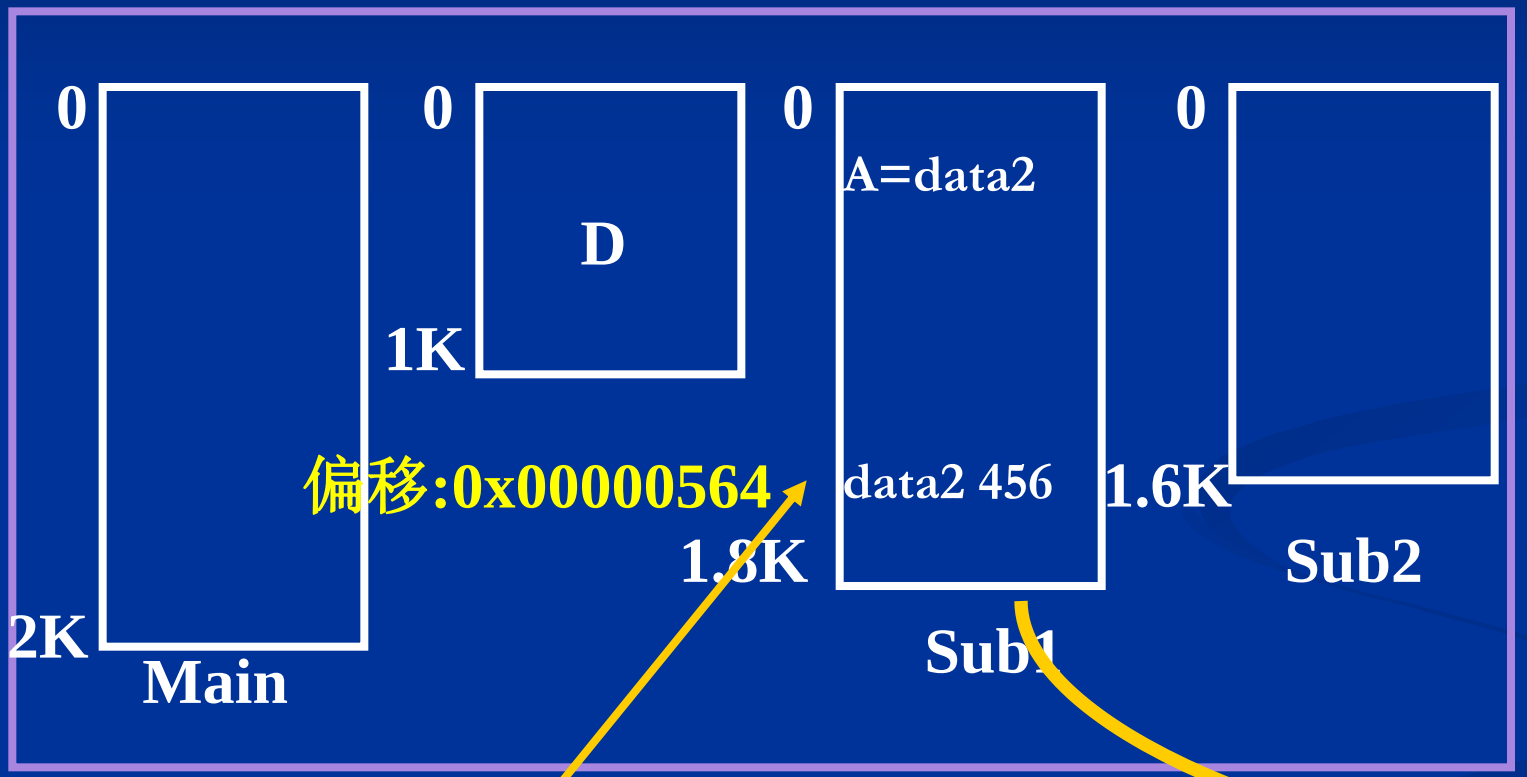
包括：段号和段地址2部分。

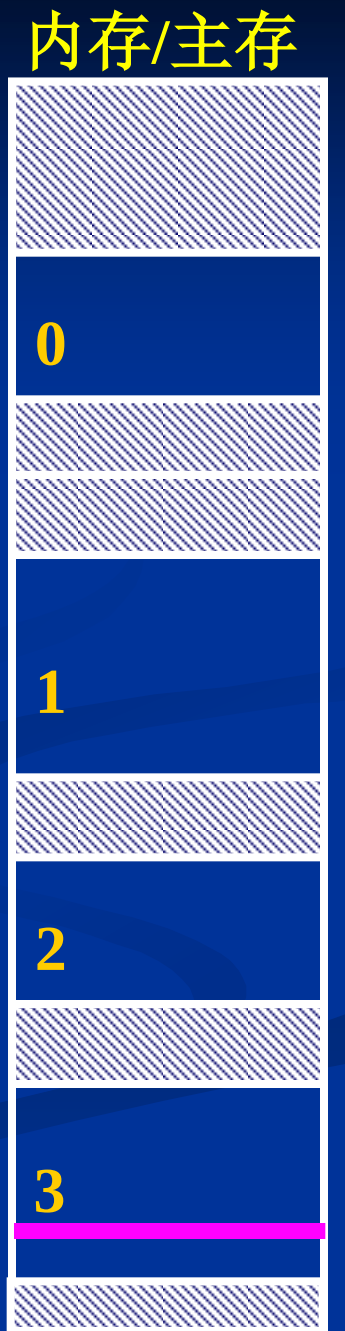




4.4.2 地址重定位

(1) 问题（假设32位地址，分段最大尺寸64K）：



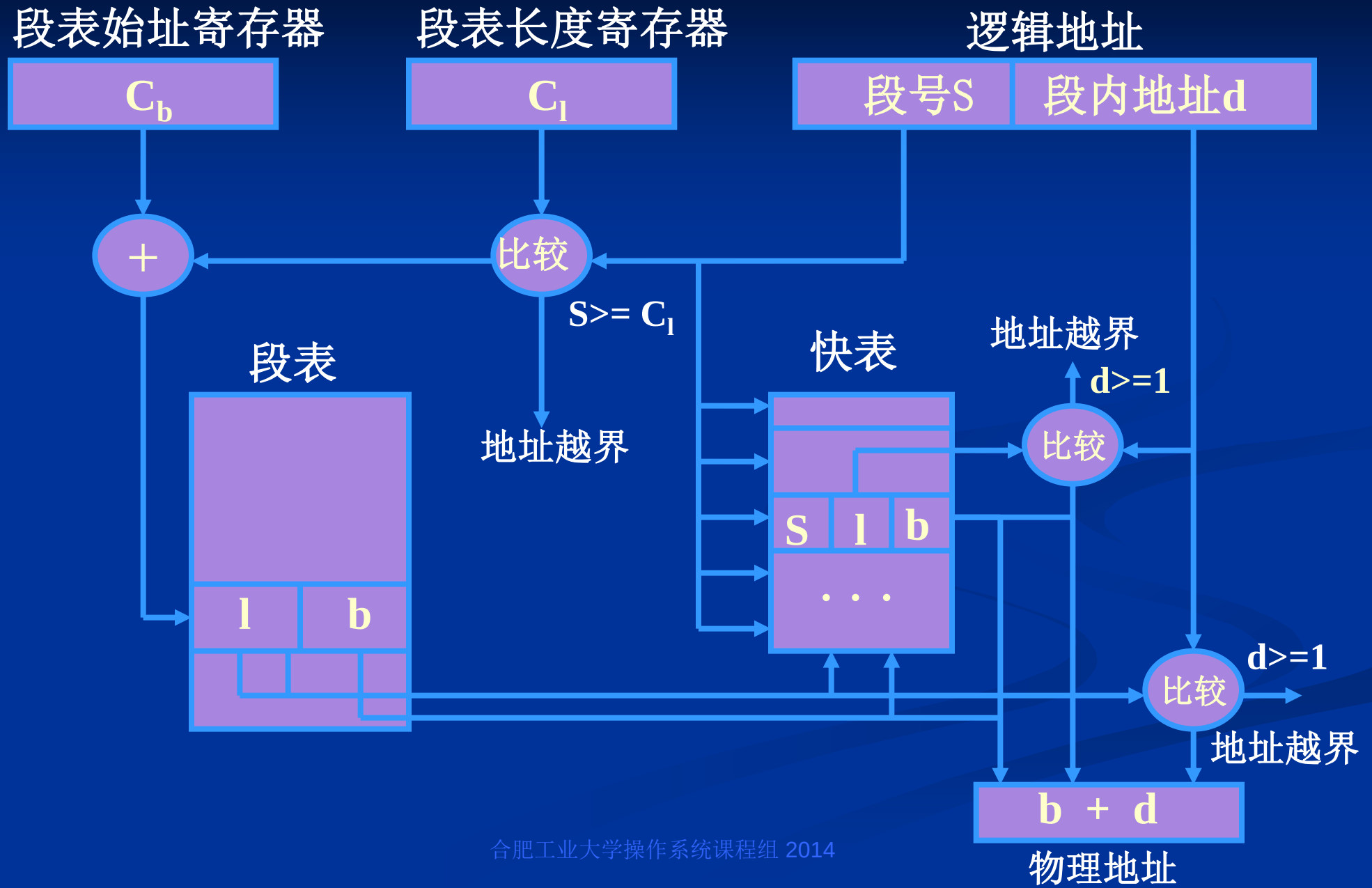


data2逻辑地址： 0x00030564

?

data2物理地址：230K+0x00000564

(2) 分段地址变换及存储保护机构



(2) 地址变换机构/地址映射机制：计算示例

假设32位地址空间，分段最大64K



data1逻辑地址：0x00030564

230K

data1物理地址：230K+0x00000564

段表：

段号	段基址	段长	存取控制
0	20K	1.6K	X
1	40K	1K	RW
2	120K	1.8K	X
3	230K	2K	X

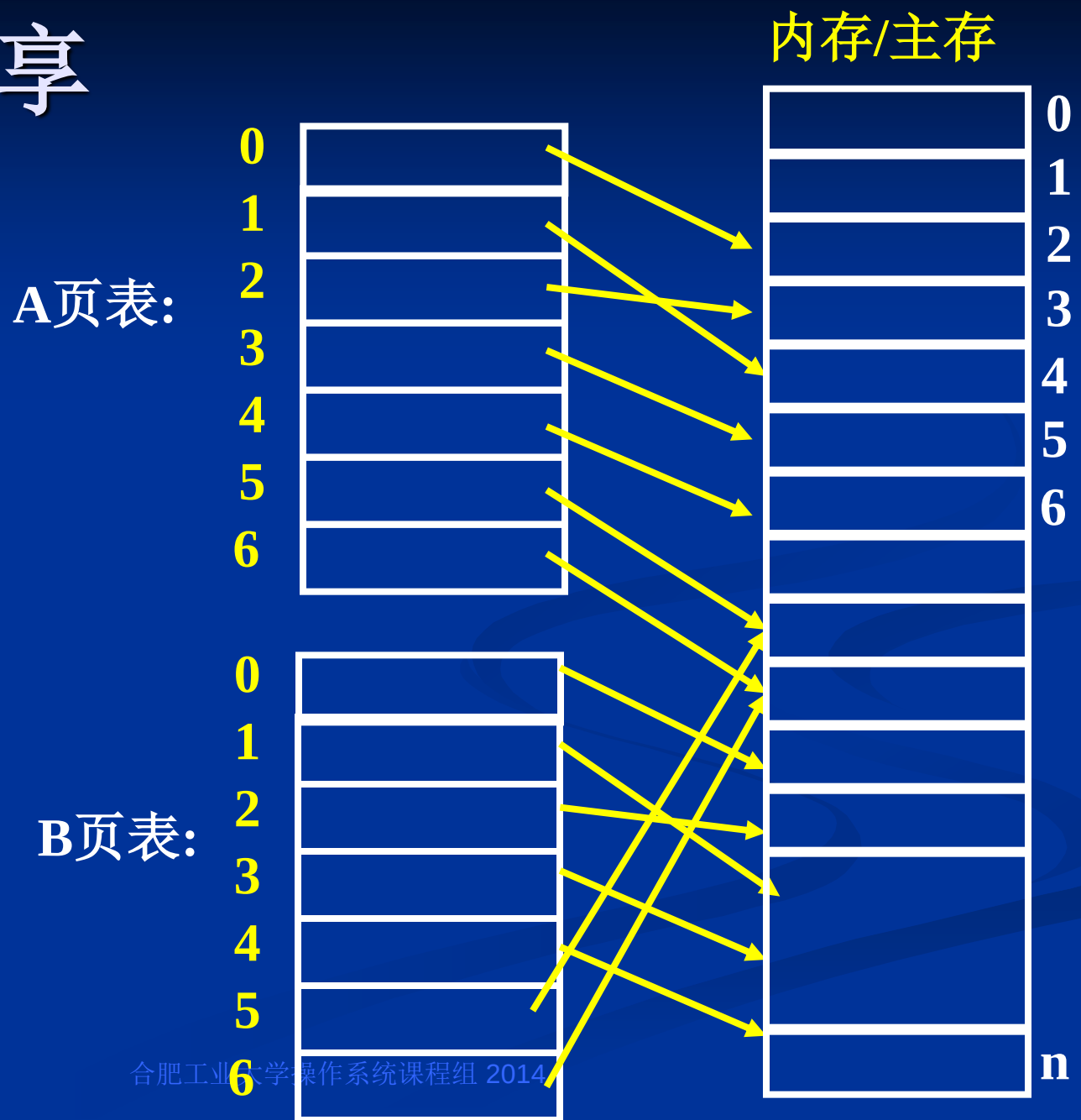
- **段表的作用**：分段系统的核心数据结构
  - 记录程序各分段所在的内存位置；
  - 支持进行地址重定位；
  - 实现分段访问控制；
  - 存储保护：限制程序在操作系统指定的内存区域内运行。

(3) 分段和分页区别

项目	分段	分页
信息单位	信息的逻辑单位	信息的物理单位
大小	不定	固定
可见性	程序员确定，可见	系统确定，程序员不可见
地址空间	二维地址空间	一维线性地址空间
信息共享保护	方便	不方便

4.4.3 信息共享

(1) 分页信息共享



4.4.3 信息共享

(2) 分段信息共享

A段表:

段号	段基址	段长	存取控制
0	230K	2K	X
1	40K	1K	RW
2	120K	1.8K	X
3	20K	1.6K	X

B段表:

段号	段基址	段长	存取控制
0	160K	2K	X
1	140K	1K	RW
2	120K	1.8K	X
3	20K	1.6K	X



合肥工业大学操作系统课程组 2014

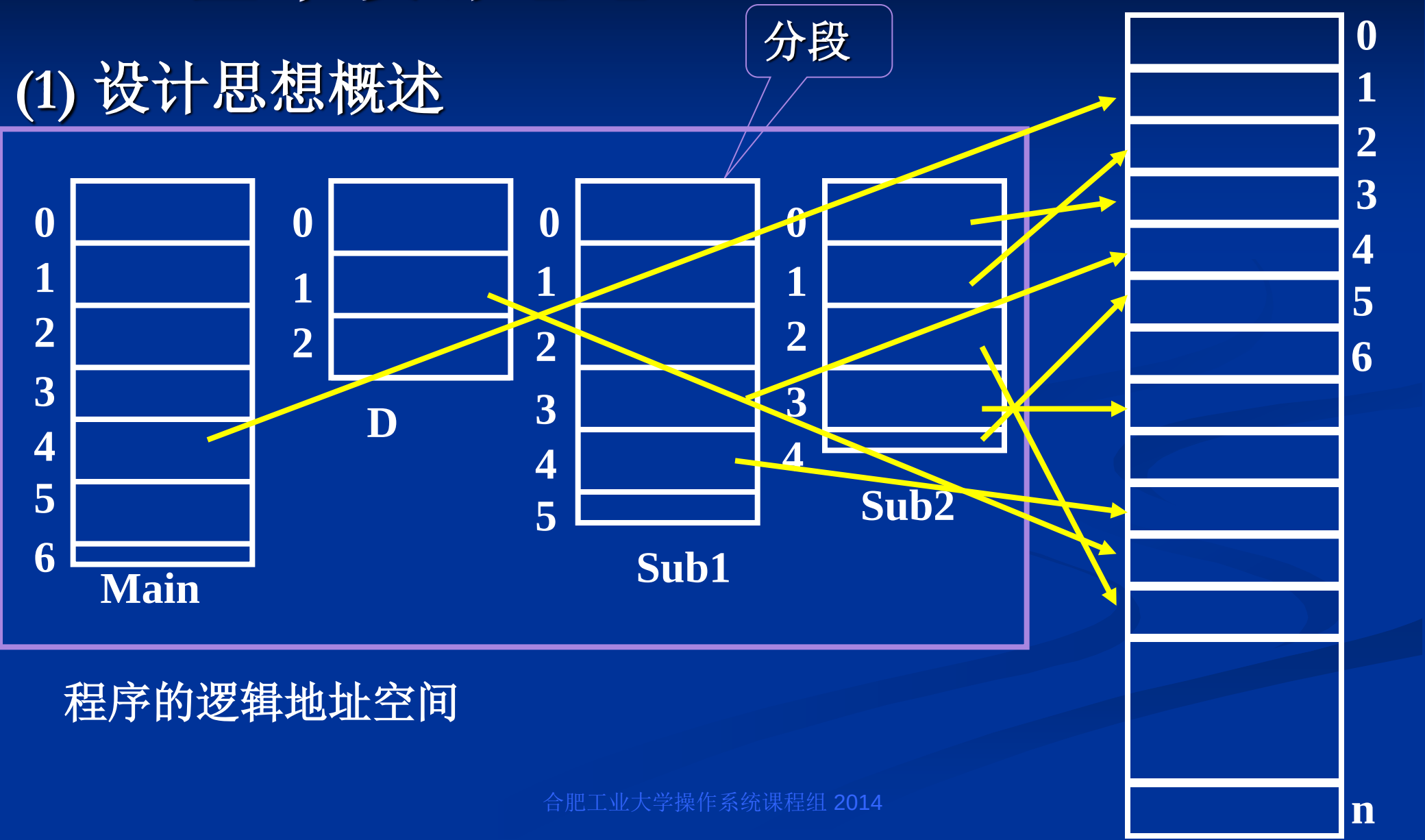
4.5 段页式存储管理方式

- 分段系统存在的问题:
  碎片（内外“零头”）问题难以解决；

- 解决方案(分段+分页)：
  - 分页系统：
    负责解决主存分配问题：内存按页分配；
  - 分段系统：
    负责解决逻辑地址空间管理问题：按段为应用程序分配逻辑地址空间；

4.5.1 基本设计思想

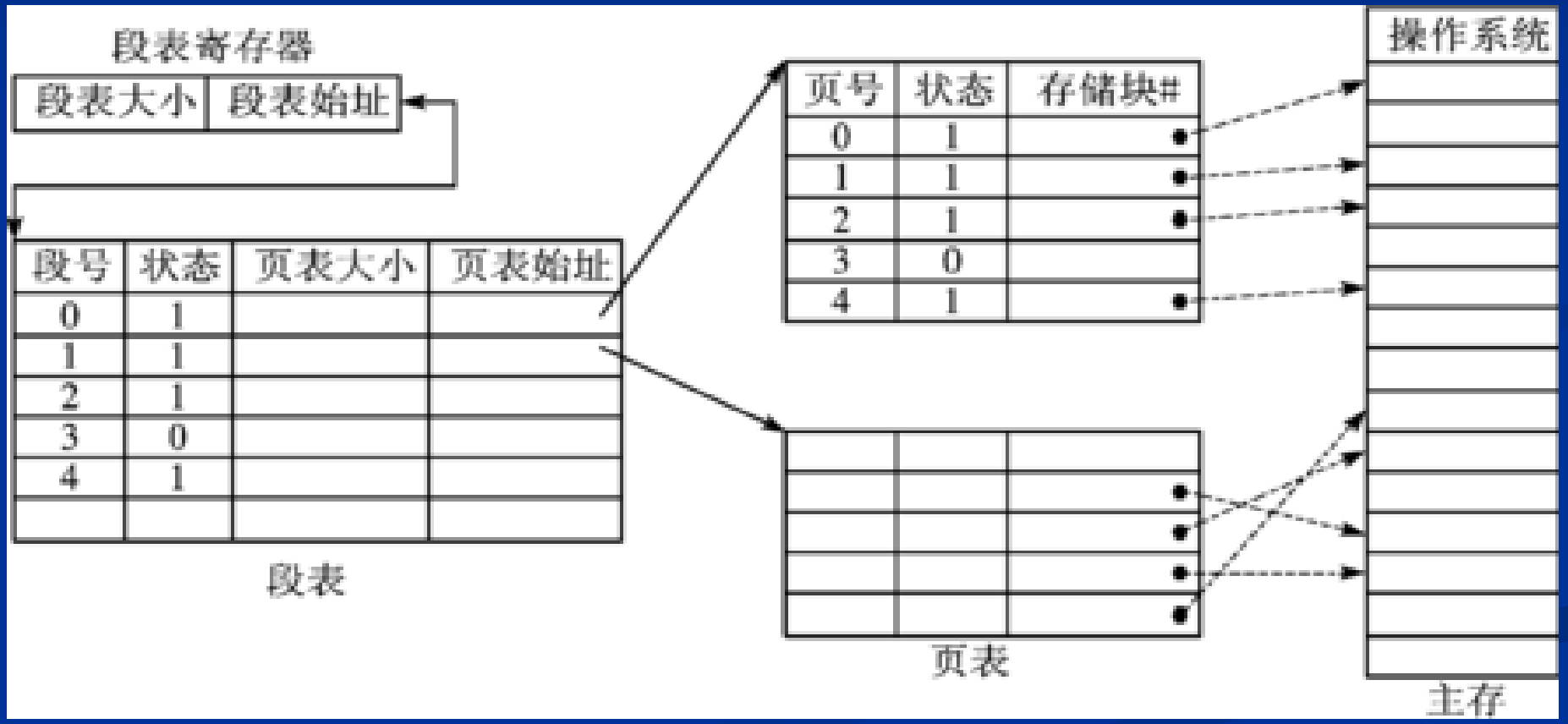
(1) 设计思想概述



程序的逻辑地址空间

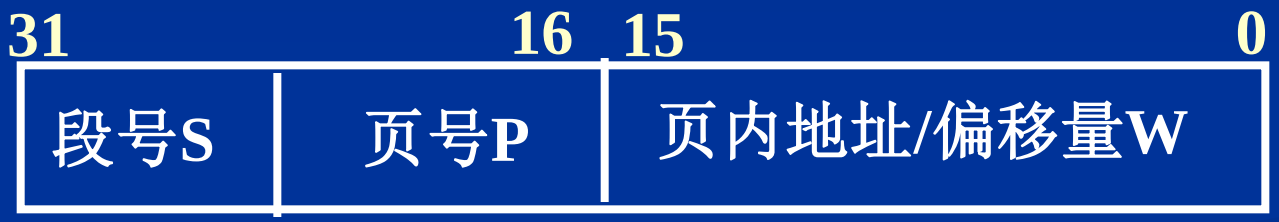
4.5.1 基本设计思想

(2) 段表和页表

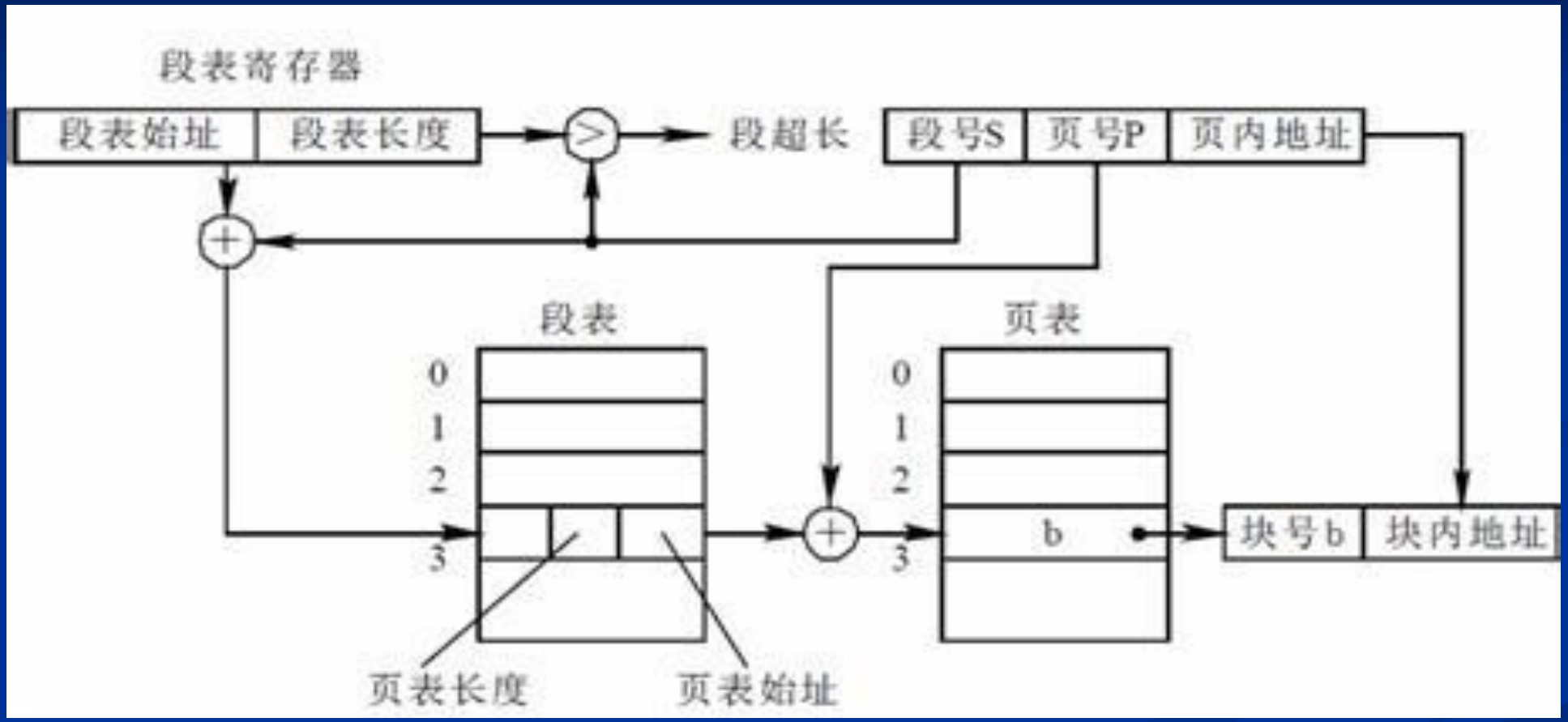


4.5.1 基本设计思想

(3) 地址结构



4.5.2 地址变换


段表寄存器
段表始址
段表长度
>
段超长
段号S
页号P
页内地址
段表
0
1
2
3
页表
0
1
2
3
b
块号b
块内地址
页表长度
页表始址

4.6 虚拟存储器的基本概念

- 存储管理的两个基本问题：
  - 大作业如何在小主存上运行；
  - 如何在给定大小主存上运行更多程序；

4.6.1 虚拟存储器的理论基础

(1) 程序运行的基本特征：局部性原理

在一段时间内一个程序的执行往往呈现出高度的局部性，表现在时间与空间两方面 [Denning P. 1968]：

- 时间局部性：

  一条指令被执行了，则在不久的将来它可能再被执行；

  在一段时间内，访问的代码范围是有限的。

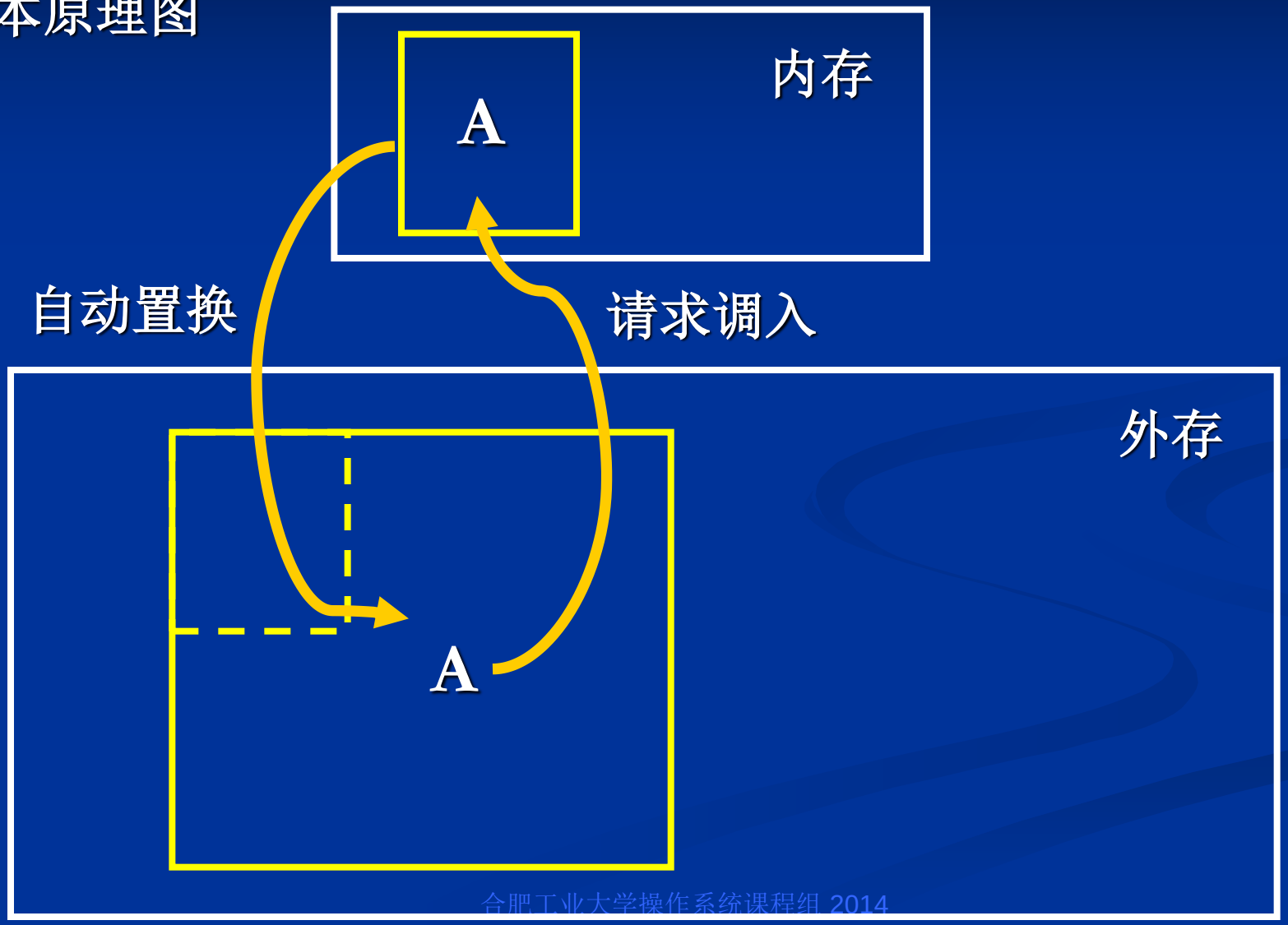
- 空间局部性：

  若某一存储单元被使用，在一定时间内，与该存储单元相邻的单元可能被使用

- 启示：是否可不将程序所有代码同时装入主存？

4.6.2 虚拟存储器概念

(1) 基本原理图



4.6.2 虚拟存储器概念

(2) 虚拟存储器的定义：

- 所谓虚拟存储器，是指具有请求调入功能和置换功能，能从逻辑上对内存容量加以扩充的一种存储器系统。
- 其逻辑容量由内存容量和外存容量之和所决定，其运行速度接近于内存速度，而每位的成本却又接近于外存。
- 虚拟存储技术是一种性能非常优越的存储器管理技术，故被广泛地应用于大、中、小型机器和微型机中。

(3) 虚拟存储器的特征：

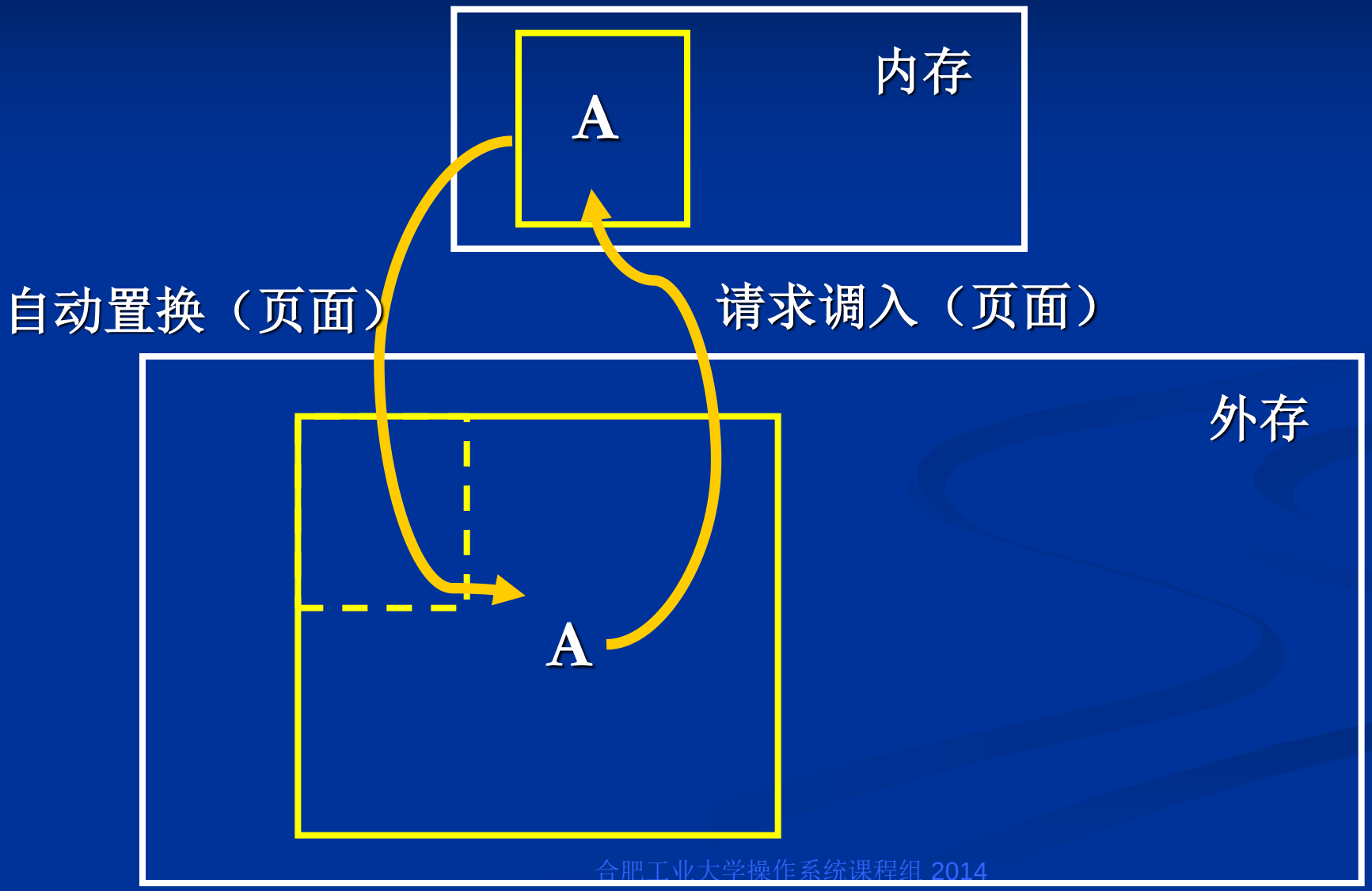
- 多次性。
- 对换性。
- 虚拟性。

4.6.2 虚拟存储器概念

(4) 虚拟存储器的实现：

- 分页请求系统（页式虚拟存储器，请求分页系统）：
  在基本分页系统基础上，增加以页面为单位的请求调入和自动置换功能 。
- 请求分段系统（段式虚拟存储器，分段请求系统）：
  在基本分段系统基础上，增加以分段为单位的请求调入和自动置换功能 。
- 段页式虚拟存储器：
  在段页式系统基础上，增加以分页为单位的请求调入和自动置换功能 。

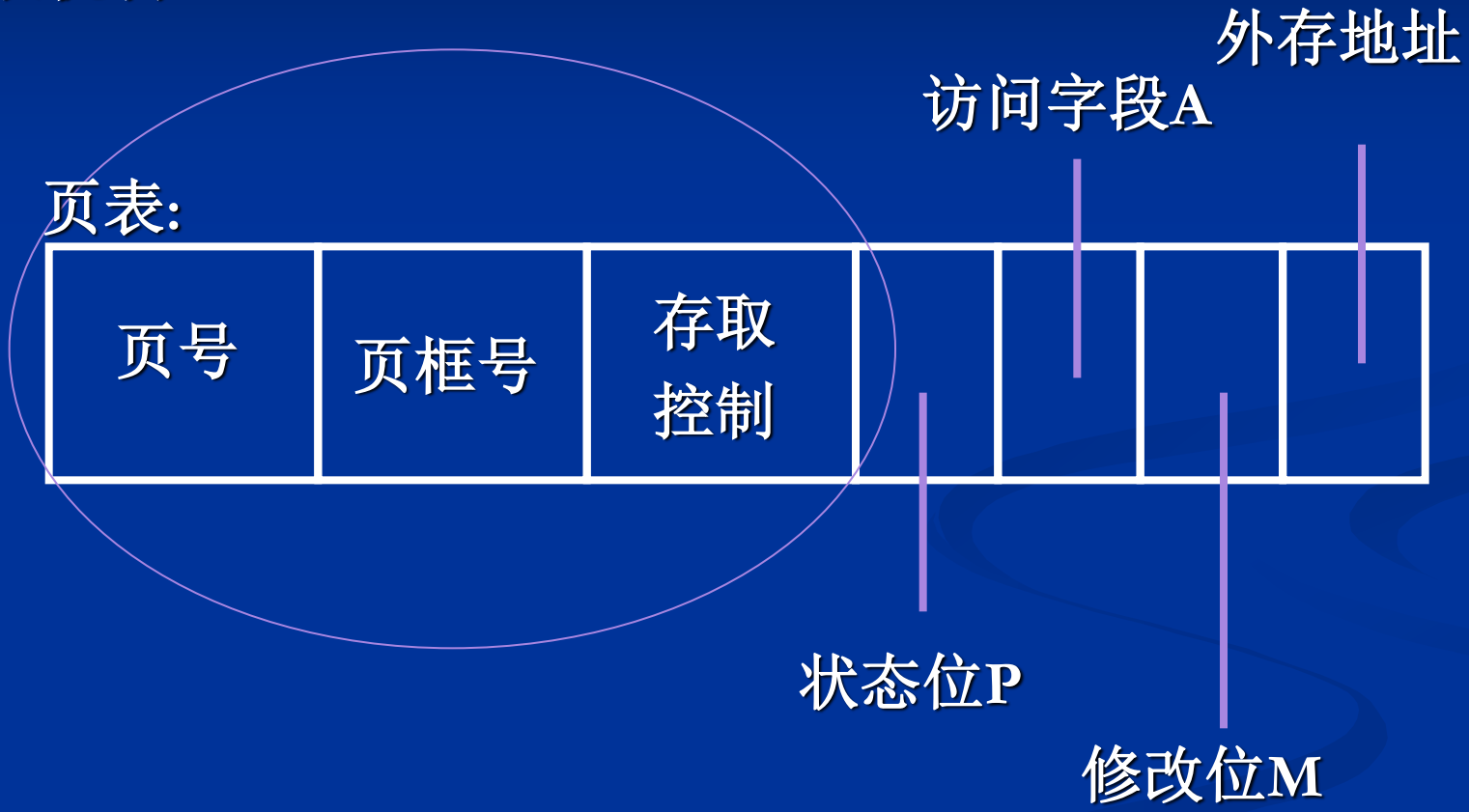
4.7 请求分页存储管理方式



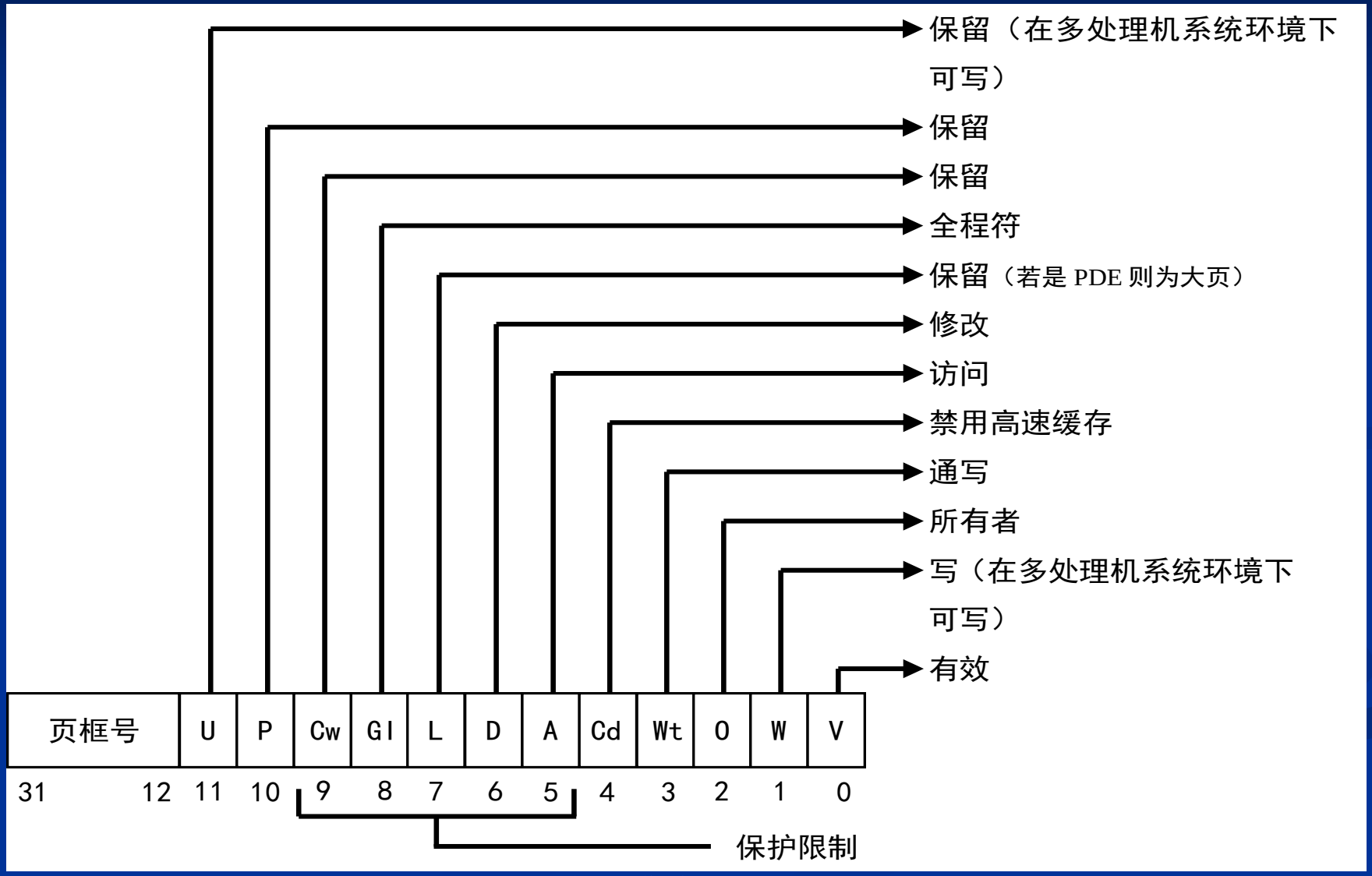
- 要解决的问题：
  - 页面现在可能不在主存了，那么如何修改页表；
  - 何时启动页面调入；
  - 地址变换；
  - 初始页面数量如何确定；
  - 如何置换页面？
  - 何时调入页面；
  - 何处调入页面；

4.7.1 请求分页中的硬件支持

(1) 页表机制



Intel x86 CPU页表



4.7.1 请求分页中的硬件支持

(2) 缺页中断机制

- 何时发出缺页中断

  MOV AX, [1000]；　CALL CS:[100]；　JNZ CS:[2000]

  CS:IP

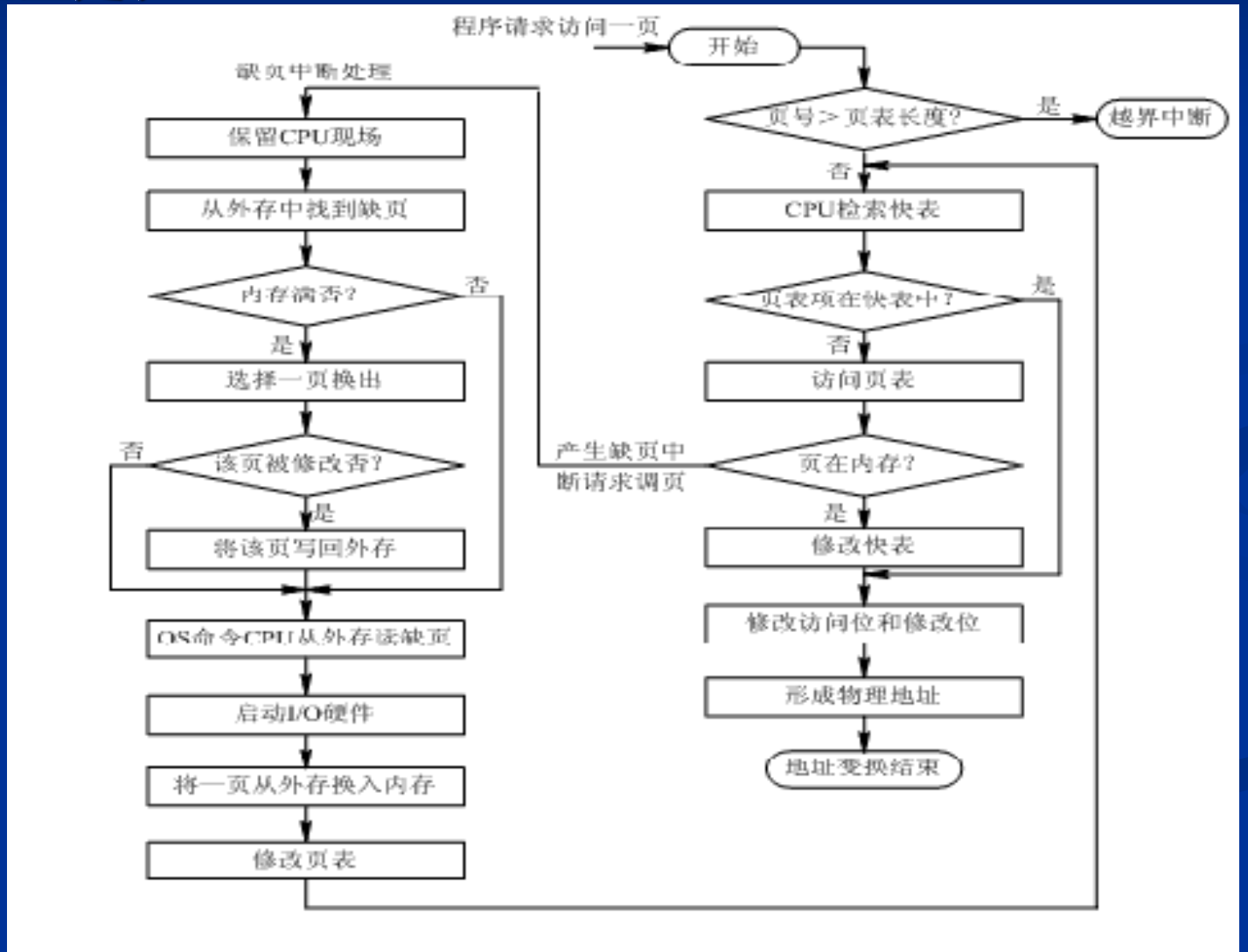
- 缺页中断
  - 指令执行期间：发出中断并响应和处理中断，返回
  - 一条指令可能发出多次缺页中断；

问题：Intel x86 CPU，为何没有下述指令？

MOV DS:[2000], CS:[1000]

4.7.1 请求分页中的硬件支持

(3) 地址重定位



4.7.2 内存分配策略和算法

(1) 最小物理块个数

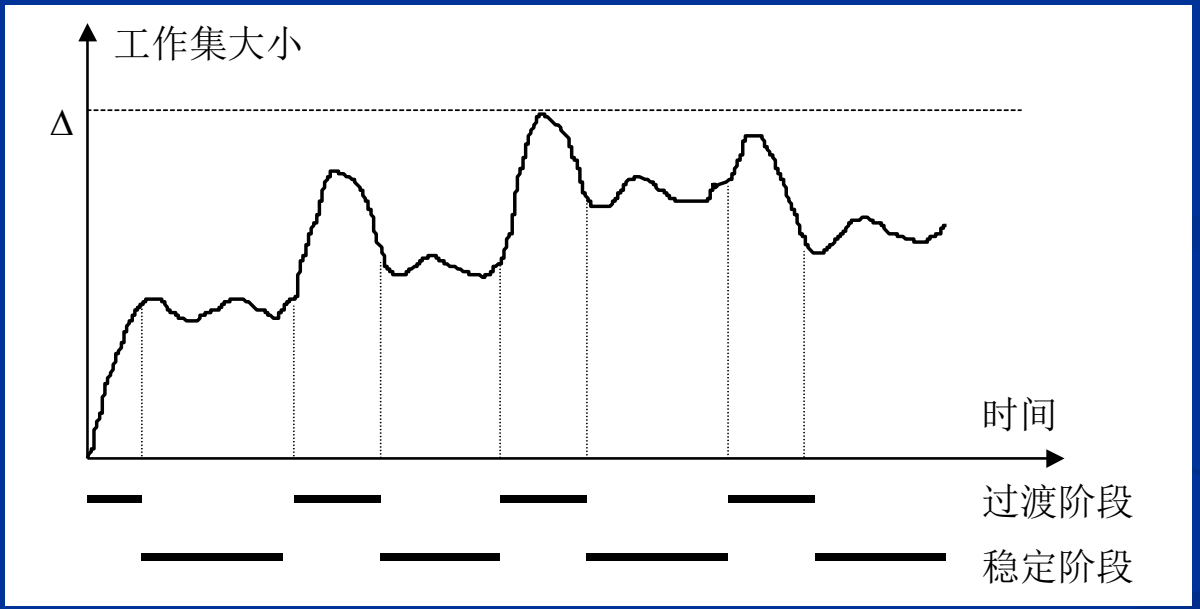
- 保证程序正常运行需要的最小物理块个数：
  - 由机器指令的结构决定；
  - 单字节指令，直接寻址：2块；
  - 单字节指令，间接寻址：3块；
  - 多字节指令，直接寻址：3块；
  - 多字节指令，间接寻址：4块；

4.7.2 物理块分配策略和算法

(2) 工作集 [Denning P.]：进程执行时需要多少物理块？

- 进程的工作集

  驻留在物理内存中的虚拟页面的子集。



工作集大小的变化：进程开始执行后，随着访问新页面逐步建立较稳定的工作集。当内存访问的局部性区域的位置大致稳定时，工作集大小也大致稳定；局部性区域的位置改变时，工作集快速扩张和收缩过渡到下一个稳定值。

4.7.2 内存分配策略和算法

(3) 物理块分配策略

- 固定分配局部置换
  - 为进程分配物理块；
  - 在进程运行期间，物理块个数不变；
  - 新页面，只能置换入该进程已经分配的物理块；
- 特点
  - 物理块个数难以指定；

4.7.2 内存分配策略和算法

(3) 物理块分配策略

- 可变分配全局置换
  - 为进程分配物理块；
  - 在进程运行期间，物理块个数随时变化；
  - 新页面，在全局范围内置换物理块；
- 特点
  - 进程之间的执行互相影响；

4.7.2 内存分配策略和算法

(3) 物理块分配策略

- 可变分配局部置换
  - 为进程分配物理块；
  - 在进程运行期间，物理块个数可变化；
  - 新页面，在该进程已经分配物理块内置换；
  - 引入缺页率，保证：

    低限阈值 <= 缺页率 <= 高限阈值
- 特点
  - 避免物理块初始设置不合理的难题；

4.7.2 内存分配策略和算法

(4) 物理块分配算法

- 平均分配
- 按比例分配
- 加权分配

4.7.3 页面策略

(1) 何时调入

- 预先调页
  - 一次调入多个页面；
  - 减少后期I/O；
- 请求式调页
  - 需要时，调入；

(2) 何处调入

- 外存对换区
- 文件区
- UNIX方式：文件区(第一次)；对换区(兑换区)；

4.7.4 页面置换算法

- 最佳置换算法(OPT算法)
- 先进先出算法(FIFO算法)
- 最久未使用页面置换算法(LRU算法)
- Clock置换算法

4.7.4 页面置换算法

(1) 最佳置换算法

- 基本思想
  - 选择以后再也不用的页面；
  - 没有的话，选择以后最长时间不用的页面；
- 实现
  - 无法实现，因为页面的访问顺序无法预知；
- 特点
  - 无法实现，仅具有理论意义；

4.7.4 页面置换算法

(2) 先进先出置换算法(FIFO)

- 基本思想
  - 基于：程序的顺序执行特点
  - 选择到达内存最早的页面，予以淘汰；
- 实现
  - 页面在内存中按时间排序；
- 特点
  - 效果不佳（程序不是严格顺序执行）；

4.7.4 页面置换算法

(3) 最近最久未使用置换算法(LRU)

- 基本思想
  - 基于：程序运行的局部性原理；
  - 选择最近以来最久未使用的页面，予以淘汰；
- 实现
  - 移动寄存器，栈；
- 特点
  - 调度性能教好；

4.7.4 页面置换算法

(3) 最近最久未使用置换算法(LRU)

- 移位寄存器实现方案
  - 每个物理块设置一个移位寄存器，初值为0；
  - 访问一次页面，高位置1；
  - 定时（100ms)将所有物理块的移位寄存器右移1位，高位补0；
  - 选择移位寄存器数值最小的物理块，淘汰其中页面

4.7.4 页面置换算法

(3) 最近最久未使用置换算法(LRU)

- 栈实现方案

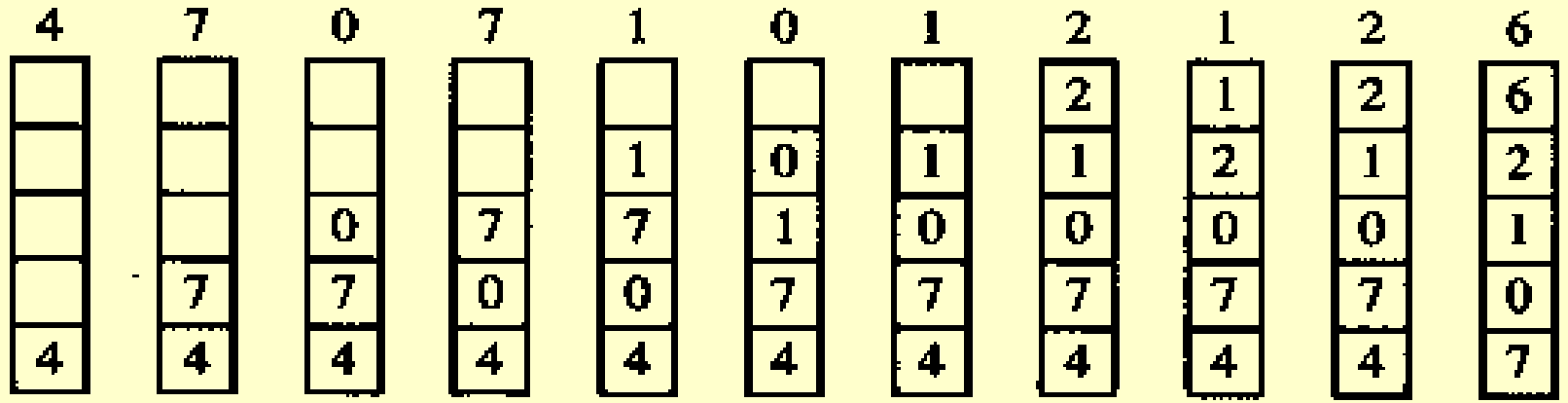


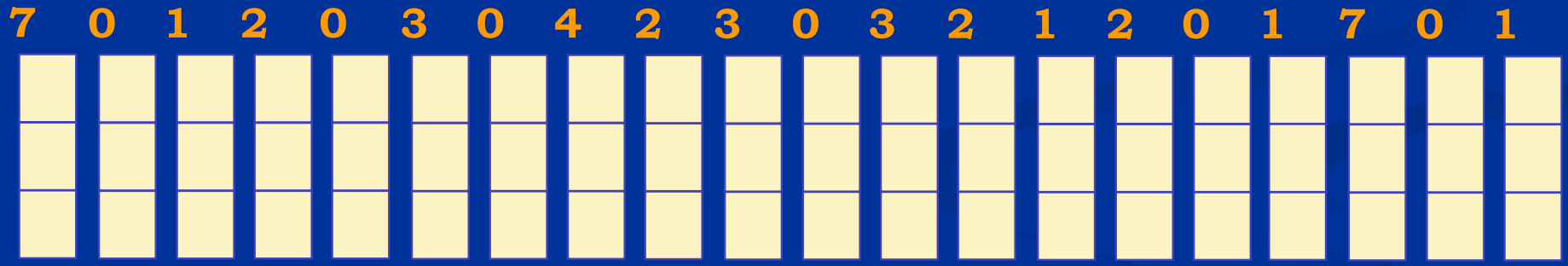
图 4－29　用栈保存当前使用页面时栈的变化情况

4.7.4 页面置换算法

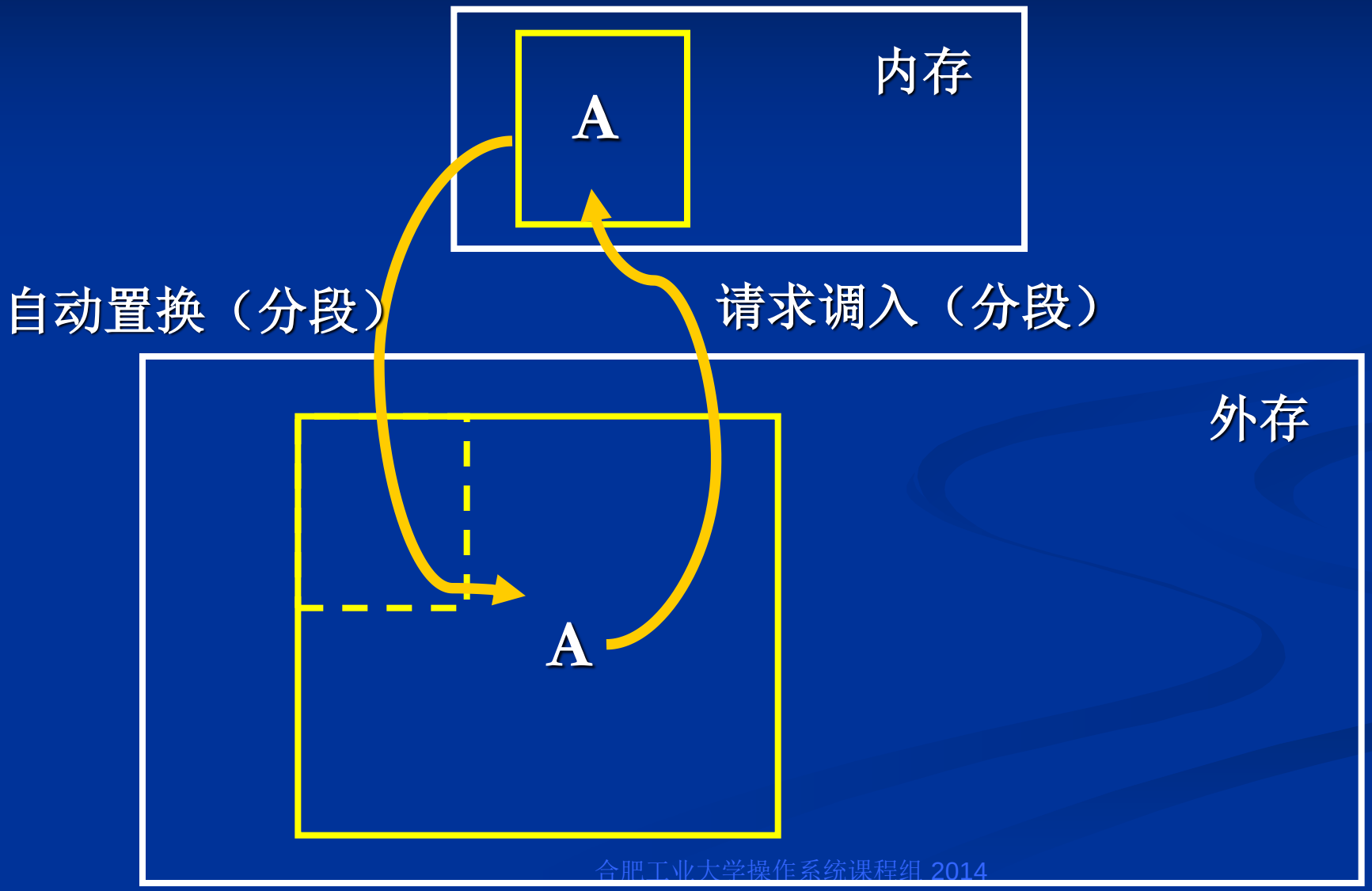
(4) 置换算法性能比较

- 实例1：

  某分页系统的页面走向如下，请分别计算三种页面调度算法的缺页率。



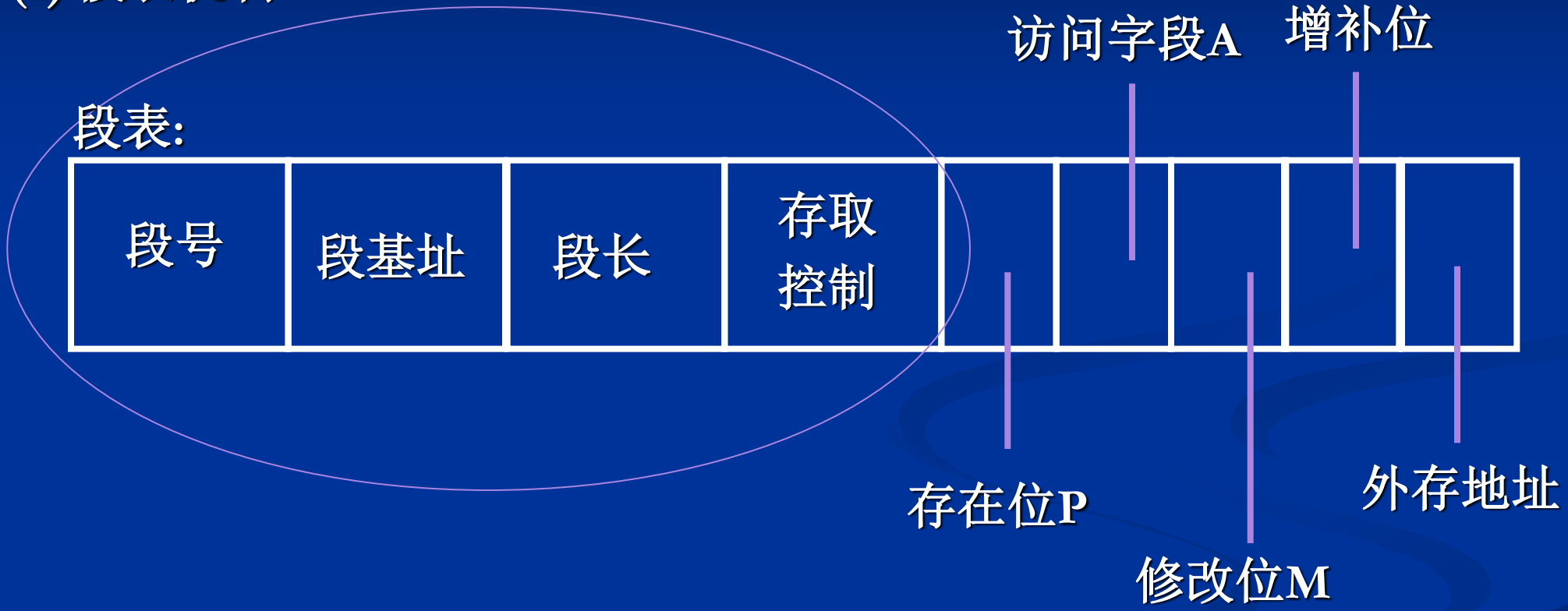
4.8 请求分段存储管理方式



- 要解决的问题：
  - 分段现在可能不在主存了，那么如何修改段表；
  - 何时启动分段调入；
  - 地址变换；
  - 如何实现分段共享？

4.8.1 请求分段中的硬件支持

(1) 段表机制



4.8.1 请求分段中的硬件支持

(2) 缺段中断机制

- 何时发出缺段中断

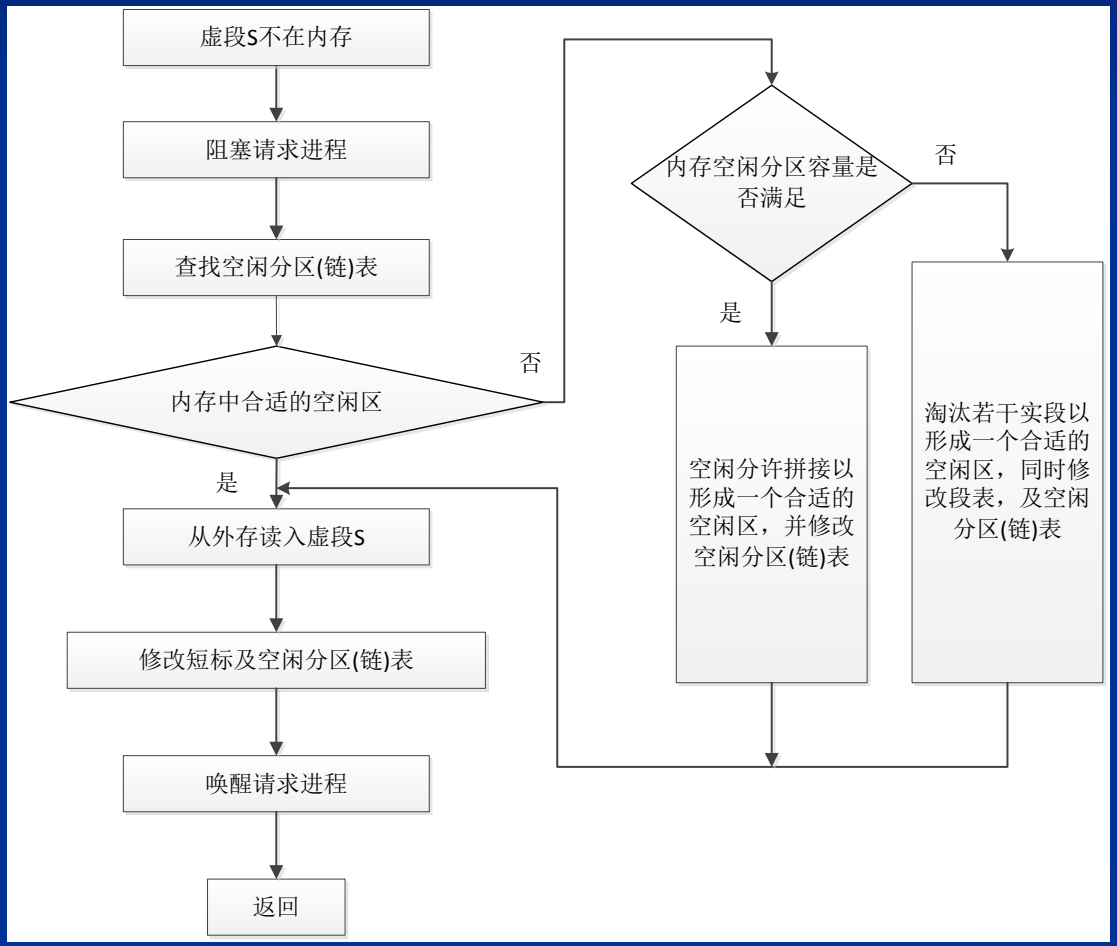
  MOV AX, [1000];　CALL CS:[100];　JNZ CS:[2000];

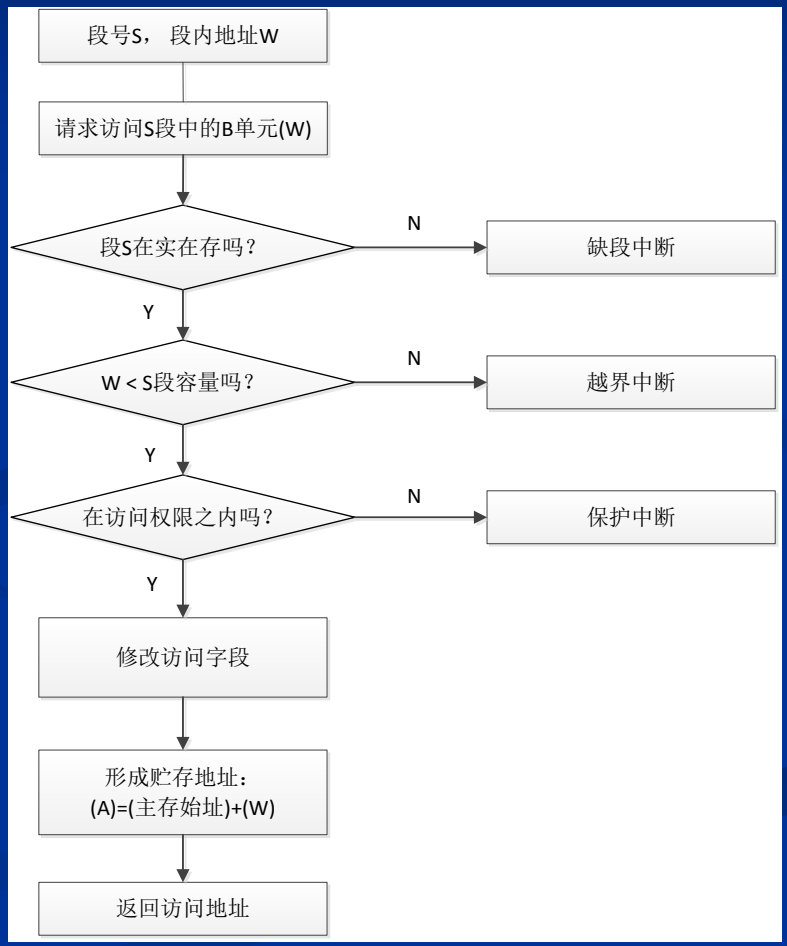
  CS:IP

- 缺段中断
  - 指令执行期间：发出中断并响应和处理中断，返回
  - 一条指令可能发出多次缺段中断；

4.8.1 请求分段中的硬件支持

(3) 地址重定位





4.8.2 分段共享与保护

(1) 数据结构

- 共享段表（所有进程共享）

  整个系统设置1张共享段表；

  记录：分段信息，共享该分段进程的信息(存取控制，进程数量count，段号)。

- 进程段表（私有段表）

  与前面所述段表相同；

注意：被共享分段可由各进程以不同方式（存取控制，段号）共享。

4.8.2 分段共享与保护

(2) 算法：共享分段的分配与回收

- 分配算法

  a) 检索共享分段，如果共享分段存在转c)；

  b)创建共享段表的段表项，填写分段信息，分配相应内存并调入分段;

  c) 记录本进程信息到共享段表的段表项,count+1;

  d) 复制共享段表的段表项相应信息（段基址、段长、存取控制）到进程段表;

- 回收算法

  a) 撤消进程时，将所有该进程共享之分段的count-1；

  b) 当某分段count =0时，回收该分段所占据内存;

本章习题

- P142(修订版教材):

  5、6、14、15、16、23